

From Source to Execution: Design and Implementation of a Statically-Typed Programming Language with Type Inference

by *The Token Trio*

Aye Khin Khin Hpone (Yolanda Lim), st125970

Applegate T. Tun Oo, st126690

Win Htut Naing, st126687

AT70.07 — Programming Languages and Compilers

Assignment 2

Instructor: Prof. Phan Minh Dung

Teaching Assistant: Akkradet Sinsamersuk

Asian Institute of Technology

School of Engineering and Technology

Thailand

April 2026

ACKNOWLEDGEMENTS

The authors would like to express their sincere gratitude to the course professor, Prof. Phan Minh Dung, for the guidance, instruction, and feedback that shaped this project. We also thank the teaching assistant, P'Akraradet Sinsamersuk, for the continued support, clarifications, and assistance provided throughout the implementation and testing process.

ABSTRACT

This report presents the design and implementation of a statically-typed interpreted programming language implemented through a compiler pipeline consisting of lexical analysis, parsing, static semantic analysis, and interpretation. The language provides four primitive types — Integer, Float, Boolean, and String — and supports arithmetic and equality expressions, variable assignment with type inference from first use, conditional and iterative control flow, user-defined functions with value parameter passing, and a built-in callable `print()` function. Types are inferred from program context rather than declared explicitly, combining static type safety with a declaration-free syntax. The implementation comprises a hand-written lexer, a recursive-descent parser that constructs an abstract syntax tree together with an inspectable parse-tree view, a static type checker that performs inference, and a tree-walking interpreter. The command-line runner and interactive desktop interface expose five labelled inspection outputs — tokens, parse tree, AST, type table, and execution output — derived from those four processing phases. The type checker implements a syntax-directed type inference algorithm over the abstract syntax tree, enforcing static type safety without explicit declarations. The report documents the formal grammar, the type inference algorithm, the system architecture, and the automated test suite of 77 tests that validate the complete implementation.

CONTENTS

CHAPTER 1

INTRODUCTION

Programming languages define the formal contract between a programmer’s intent and machine execution. Building a language processor from source text through to evaluated output requires a coherent pipeline of components, each of which must correctly transform its input before passing it downstream. Lexical analysis identifies atomic tokens; parsing imposes hierarchical structure; static type checking validates semantic correctness before execution; and interpretation evaluates the validated program (??). When these stages are connected cleanly, errors are caught at the earliest possible point, and the overall system remains tractable to reason about, test, and extend.

This report presents the design and implementation of a statically-typed interpreted language that realises this pipeline in full. The language supports four primitive types — Integer, Float, Boolean, and String — and provides arithmetic and equality expressions, assignment, conditional and iterative control flow, user-defined functions with value parameter passing, and a built-in callable `print()` function. Types are inferred from program context rather than declared explicitly, combining the safety of static typing with the conciseness of a declaration-free syntax. The implementation comprises three principal components developed in pipeline order: the lexer and symbol table, the recursive-descent parser with parse-tree and abstract-syntax-tree views, and the type checker and interpreter.

The system can be invoked from the command line or through an interactive desktop interface, both exposing five inspection views — tokens, parse tree, AST, inferred type table, and execution output. The parse tree supports reporting and debugging, while the AST remains the executable intermediate representation consumed by the type checker and interpreter. An automated regression suite of 77 tests verifies correctness across all stages and language features. The report explicitly distinguishes source-level lexeme strings, token types emitted by the lexer, and CFG terminals consumed by the parser; Section ?? demonstrates this separation concretely through an annotated example.

The main contributions of this project are:

- Design of a statically-typed programming language supporting Integer, Float, Boolean, and String types without operator overloading.
- Implementation of a hand-written lexer and a recursive-descent parser with explicit parse-tree and abstract syntax tree generation.
- Development of a syntax-directed type inference and static type-checking system without explicit type declarations.
- Separation of integer and floating-point arithmetic operators following the Caml Light convention to eliminate semantic ambiguity (?).
- Construction of a complete end-to-end execution pipeline consisting of lexical analysis, parsing, semantic analysis, and interpretation.
- Development of both command-line and graphical interfaces exposing all compiler pipeline stages for inspection and debugging.
- Validation of the implementation through automated regression testing and semantic error verification.

The remainder of this report is organised as follows. Chapter 2 presents the language design, token specification, grammar, and operator precedence rules. Chapters 3 and 4 cover the front end in pipeline order: lexical analysis and the symbol table, followed by recursive-descent parsing and the AST. Chapter 5 explains the type inference algorithm and semantic rules. Chapter 6 presents the system architecture and implementation, including the end-to-end pipeline, project layout, and technology stack. Chapter 7 discusses testing and verification, and Chapter 8 concludes the report. A supplementary walkthrough document traces the complete pipeline end-to-end for a concrete factorial program, covering all four stages from lexical analysis through to interpreted output.

CHAPTER 2

LANGUAGE DESIGN AND GRAMMAR

2.1 Types

The language provides four primitive types, chosen to cover the core domains of numeric computation, logical control flow, and text manipulation while keeping the type system tractable and the semantics well-defined.

2.1.1 *Integer*

The Integer type represents whole numbers such as 0, 7, and 42. Integer literals are written as one or more decimal digits without quotation marks or a decimal point.

2.1.2 *Float*

The Float type represents real-valued numbers written with a decimal point, such as 3.14 or 0.5. Float values are used exclusively with the dot-suffixed arithmetic operators ($+. , -. , *. , /. ,$), following the Caml Light convention. Integer division ($/$) performs floor division and returns Integer: for example, $10 / 3$ evaluates to 3 : Integer.

The language intentionally does not perform implicit numeric promotion between Integer and Float values. This choice follows directly from the design requirement of “no overloading with the usual precedence of multiplication and division over addition and subtraction.” In a language where a single $+$ operator accepts both $(Integer, Integer) \rightarrow Integer$ and $(Integer, Float) \rightarrow Float$, the operator effectively carries multiple type signatures, since the compiler must inspect operand types to determine the result type. This constitutes operator overloading, even when described as “numeric promotion” or “widening coercion.” To eliminate that ambiguity, the language follows the Caml Light convention of using separate operator tokens for integer and floating-point arithmetic: $+$, $-$, $*$, and $/$ operate only on Integer operands, whereas $+. , -. , *. ,$ and $/. ,$ operate only on Float operands. Mixed-type arithmetic such as $1 + 2.5$ is therefore rejected by the type checker. Under this design, each operator token uniquely determines both operand and result types, so operator meaning never depends on post hoc type resolution.

2.1.3 Boolean

The Boolean type represents logical truth values. The two Boolean literals are `true` and `false`. Boolean values are used in conditions and as results of comparison expressions.

2.1.4 String

The String type represents sequences of characters enclosed in double quotes, for example `"hello"`. Strings can be assigned to variables and printed through the built-in `print()` function. The surrounding quotes are stripped by the parser when the `Literal` node is created, so the value stored internally is the bare character sequence without surrounding quotes. At runtime, the built-in `print()` function re-adds double quotes around string values, so `print("plc")` outputs `"plc" : String`.

2.1.5 Void

Although the language exposes exactly four programmer-visible types, the implementation introduces a fifth value, `Void`, within the `LanguageType` enumeration. Its sole purpose is to provide a well-defined return type for functions that contain no `return` statement, allowing the symbol table to record a consistent entry for every function regardless of whether it produces a value. `Void` cannot appear in source code and is never visible to the programmer; it is a bookkeeping device used internally by the type checker. Therefore, `Void` does not extend the programmer-visible source-level type system.

2.2 Static Typing

The language employs static typing, meaning that type errors are detected at type-checking time rather than at runtime. No explicit variable type declarations are required. Instead, the type checker infers types from literals, assignments, operations, function calls, and return statements. Once a variable type is established, later assignments must remain consistent with that type. For example, the following program is rejected at the type-checking stage before any execution occurs:

```
1 x = 5;  
2 x = "hello";
```

The type checker rejects the second assignment with:

```
1 [line 2, col 1] TypeError: Cannot assign String to variable 'x' of type Integer
```


This contrasts with dynamically-typed languages, where the same program would run without error until (or unless) a type-dependent operation was attempted.

2.3 Lexeme Grammar

The **lexeme grammar** operates at the character level: it defines the regular expressions the lexer uses to scan raw source characters into lexeme strings. The notation below uses + (one or more), * (zero or more), | (alternation), and [. . .] (character class).

```
1 (* Character classes *)
2 digit ::= [0-9]
3 letter ::= [a-zA-Z] | '_'
4
5 (* Literals *)
6 INT_LITERAL ::= digit+
7 FLOAT_LITERAL ::= digit+ '.' digit+
8 BOOL_LITERAL ::= 'true' | 'false'
9 STRING_LITERAL ::= '"' [^"\n]* '"'
10
11 (* Identifiers and keywords *)
12 IDENTIFIER ::= letter (letter | digit)*
13             -- where the lexeme is not in the KEYWORDS set
14 KEYWORD ::= 'if' | 'else' | 'while' | 'def' | 'return'
15
16 (* Arithmetic operators -- integer and float variants *)
17 PLUS ::= '+' PLUS_DOT ::= '+.'
18 MINUS ::= '-' MINUS_DOT ::= '-.'
19 STAR ::= '*' STAR_DOT ::= '*.'
20 SLASH ::= '/' SLASH_DOT ::= '/.'
21
22 (* Comparison and assignment *)
23 EQUAL_EQUAL ::= '=='
24 BANG_EQUAL ::= '!='
25 ASSIGN ::= '='
26
27 (* Delimiters *)
28 LPAREN ::= '(' RPAREN ::= ')'
29 LBRACE ::= '{' RBRACE ::= '}'
30 COMMA ::= ',' SEMICOLON ::= ';'

```

Maximal-munch rule. The lexer applies the maximal-munch (longest-match) principle: at each position it consumes the longest character sequence that matches any rule. This ensures, for example, that == is scanned as one EQUAL_EQUAL token rather than two ASSIGN tokens, and that +. is scanned as one PLUS_DOT token rather than a PLUS followed by a stray dot (unless the . is followed by a digit, in which case it starts a FLOAT_LITERAL and the preceding + is emitted as PLUS).

Keyword and boolean priority. IDENTIFIER and KEYWORD share the same character-level pattern. The lexer resolves ambiguity by scanning an identifier-shaped lexeme first, then consulting the KEYWORDS map. If the lexeme matches a keyword entry (including `true` and `false`, which map to `BOOL_LITERAL`), the keyword token type is emitted; otherwise an IDENTIFIER token is produced. The name `print` is not a keyword and is therefore always scanned as an IDENTIFIER.

2.4 Token Grammar

The **token grammar** maps each lexeme class produced by the lexeme grammar to a named token type. Table ?? lists all token types recognised by the language together with their categories; the lexeme patterns are those defined formally in Section ??.

Table 2.1. Token Grammar: token types and their categories

Token Name	Category	Lexeme(s)
INT_LITERAL	Integer literal	one or more decimal digits, e.g. 0, 10, 42
FLOAT_LITERAL	Float literal	digits with decimal point, e.g. 3.14, 20.5
BOOL_LITERAL	Boolean literal	true, false
STRING_LITERAL	String literal	double-quoted text, e.g. "hello"
IDENTIFIER	Identifier	letter or _ followed by letters, digits, or _
IF	Keyword	if
ELSE	Keyword	else
WHILE	Keyword	while
DEF	Keyword	def
RETURN	Keyword	return
PLUS	Integer arithmetic	+
MINUS	Integer arithmetic	-
STAR	Integer arithmetic	*
SLASH	Integer arithmetic	/
PLUS_DOT	Float arithmetic	+.
MINUS_DOT	Float arithmetic	-.
STAR_DOT	Float arithmetic	*.
SLASH_DOT	Float arithmetic	/.
EQUAL_EQUAL	Comparison	==
BANG_EQUAL	Comparison	!=
ASSIGN	Assignment	=
LPAREN	Delimiter	(
RPAREN	Delimiter	)
LBRACE	Delimiter	{
RBRACE	Delimiter	}
COMMA	Delimiter	,
SEMICOLON	Delimiter	;
EOF	End of input	(end of source)

2.5 Context-Free Grammar

The language grammar can be described by the context-free grammar $G = (V_N, V_T, P, S)$, where V_N is the set of non-terminals, V_T is the set of terminals, P is the set of production rules, and S is the start symbol program. The grammar is implemented through recursive-descent parsing functions. For readability, the presentation below mixes classical CFG productions with EBNF shorthand in selected rules; both notations correspond to the same parser implementation.

Notation convention. Uppercase identifiers (e.g. IF, LPAREN, INT_LITERAL) are *terminal token types* exactly as defined in the Token Grammar (Table ??). They represent a named token emitted by the lexer, not the raw lexeme string. For example, the terminal IF matches the

lexeme `if` (via the lexer rule `IF ::= 'if'`), but the grammar rule itself references the *token type*, not the quoted character sequence. **Lowercase** identifiers (e.g. `if_stmt`, `block`, `expr`) are non-terminal symbols.

2.5.1 From input string to CFG terminals

The implementation maintains three distinct layers, which must not be mixed:

input source string $\rightarrow$ lexeme strings $\rightarrow$ token types $\rightarrow$ CFG productions

```
1 Input source string:
2 if (x == 0) { print(x); } else { print(y); }
```

```
1 Lexeme sequence:
2 if ( x == 0 ) { print ( x ) ; } else
3 { print ( y ) ; }
```

```
1 Token sequence:
2 IF LPAREN IDENTIFIER EQUAL_EQUAL INT_LITERAL RPAREN
3 LBRACE IDENTIFIER LPAREN IDENTIFIER RPAREN SEMICOLON RBRACE
4 ELSE
5 LBRACE IDENTIFIER LPAREN IDENTIFIER RPAREN SEMICOLON RBRACE
```

The key mappings are: `'if'` $\rightarrow$ `IF`, `'else'` $\rightarrow$ `ELSE`, `'('` $\rightarrow$ `LPAREN`, `')'` $\rightarrow$ `RPAREN`, `'{'` $\rightarrow$ `LBRACE`, `'}'` $\rightarrow$ `RBRACE`. The lexeme `print` is *not* a keyword in this language, so it maps to `IDENTIFIER`, not to a dedicated `PRINT` token.

```
1 CFG productions:
2 if_stmt -> IF LPAREN expr RPAREN block else_part
3 else_part -> ELSE block | epsilon
4 block -> LBRACE stmt_list RBRACE
```

This example shows why a CFG should use token names such as `IF` and `ELSE` as terminals, rather than quoted source strings such as `"if"` and `"else"`. A complete end-to-end pipeline trace applying these distinctions to a concrete factorial program — covering lexical analysis, tokenisation, CFG parsing, type inference, and interpretation — is provided in the supplementary walkthrough document.

2.5.2 Program Structure

```
1 program -> stmt_list EOF
2 statement -> assignment
3           | if_stmt
4           | while_stmt
```

```

5      | func_def
6      | return_stmt
7      | expr SEMICOLON
8 stmt_list -> statement stmt_list | epsilon

```

2.5.3 Expressions

An **expression** is any syntactic construct that evaluates to a value of one of the four programmer-visible types: Integer, Float, Boolean, or String. Concretely, an expression is one of:

- a literal value (INT_LITERAL, FLOAT_LITERAL, BOOL_LITERAL, or STRING_LITERAL);
- a variable reference (IDENTIFIER);
- a function call (IDENTIFIER followed by a parenthesised argument list);
- a unary arithmetic negation (MINUS or MINUS_DOT applied to an expression);
- a binary arithmetic operation (addition, subtraction, multiplication, or division, in integer or float variants);
- a comparison (EQUAL_EQUAL or BANG_EQUAL) between two arithmetic expressions, yielding a Boolean; or
- a parenthesised expression.

Expressions are the only constructs that carry a type; statements do not produce values. The grammar rules below define the precise structure and operator-precedence hierarchy of expressions:

```

1 expr -> comparison | additive
2 comparison -> additive (EQUAL_EQUAL | BANG_EQUAL) additive
3 additive -> term ((PLUS | MINUS | PLUS_DOT | MINUS_DOT) term)*
4 term -> factor ((STAR | SLASH | STAR_DOT | SLASH_DOT) factor)*
5 factor -> MINUS factor (* integer unary negation *)
6           | MINUS_DOT factor (* float unary negation *)
7           | INT_LITERAL
8           | FLOAT_LITERAL
9           | STRING_LITERAL
10          | BOOL_LITERAL
11          | IDENTIFIER
12          | IDENTIFIER LPAREN args? RPAREN
13          | LPAREN expr RPAREN

```

The comparison non-terminal is introduced so that the parse-tree renderer can label equality and inequality expressions explicitly. When an `expr` contains `no ==` or `no !=`, it expands directly to an additive; otherwise, it expands to a comparison node whose two operands are themselves additive sub-trees. Comparisons remain non-associative (one comparison per expression) and produce a `Boolean` result.

The operators `+`, `-`, `*`, and `/` operate exclusively on `Integer` operands. The dot-suffixed operators `+. , -. , *. ,` and `/.` operate exclusively on `Float` operands. This follows the Caml Light convention of using distinct operator tokens for integer and float arithmetic, eliminating the need for implicit numeric promotion. Unary negation is similarly split: `-x` negates an `Integer` (represented internally as the equivalent subtraction form `0 - x`), and `-.x` negates a `Float` (represented internally as `0.0 -. x`).

Integer division and division by zero. `/` is floor (integer) division: both operands must be `Integer`, the result is `Integer`, and the value is $\lfloor a/b \rfloor$. For example, `7 / 2` evaluates to `3 : Integer`. Float division uses the distinct operator `/.` and requires two `Float` operands. Division by zero (`x / 0` or `x /. 0.0`) is detected at runtime and raises a `RuntimeError`.

2.5.4 Boolean Expressions

Comparison operators `==` and `!=` are the lowest-precedence binary operators and produce `Boolean` values. As they are part of the general `expr` rule, `Boolean` expressions may appear both as control-flow conditions and as ordinary expressions. Their inferred type is `Boolean`, allowing comparison results to be assigned to variables, passed as arguments, or printed:

```
1 flag = (x == y);  
2 print(a != b);
```

Here `flag` is inferred as `Boolean`, and `print(a != b)` outputs either `true : Boolean` or `false : Boolean`.

Strict arithmetic operands. The specification states that `==` and `!=` operate between two arithmetic expressions. In this language, “arithmetic” is interpreted strictly as `Integer` and `Float`, consistent with the no-overloading rule. Both operands must also share the same numeric type, with no implicit coercion. Therefore `Integer == Integer` and `Float == Float` are well-typed; `Integer == Float`, `Boolean == Boolean`, and `String == String` are all rejected at type-check time.

2.5.5 Statements

```
1 assignment -> IDENTIFIER ASSIGN expr SEMICOLON
2 if_stmt -> IF LPAREN expr RPAREN block else_part
3 else_part -> ELSE block | epsilon
4 while_stmt -> WHILE LPAREN expr RPAREN block
5 return_stmt -> RETURN expr SEMICOLON
6 block -> LBRACE stmt_list RBRACE
7 stmt_list -> statement stmt_list | epsilon
```

Design note: print as a built-in callable

`print` is implemented as a normal function-call expression rather than as a dedicated statement form. Lexically, `print` is tokenised as an `IDENTIFIER`; syntactically, `print(x)` is parsed through the same `FunctionCall` path used for user-defined calls. Semantically it is a unary built-in of type $T \rightarrow \text{Void}$: it accepts one argument of any programmer-visible type, emits value : Type at runtime, and returns the internal `Void` type. As with any void-returning call, `print(...)` may be used as a standalone expression statement but not as the right-hand side of an assignment. `Void` is internal bookkeeping only: it never appears as a source-level type annotation or source-level type literal.

2.5.6 Function Definitions and Calls

```
1 func_def -> DEF IDENTIFIER LPAREN params RPAREN block
2 params -> IDENTIFIER param_tail | epsilon
3 param_tail -> COMMA IDENTIFIER param_tail | epsilon
4 args -> expr arg_tail | epsilon
5 arg_tail -> COMMA expr arg_tail | epsilon
```

The parser implementation accepts the equivalent EBNF shorthand with optional parameter and argument lists. Concretely, this means the grammar allows both `def f() {}` and `f()`. The corresponding parse-tree renderer makes the empty case explicit by emitting a single `epsilon` leaf as the only child of an empty `params` or `args` node, so an empty parameter or argument list is still labelled in the tree.

Function calls use value parameter passing, that is, call-by-value. The argument values are evaluated before the call and copied into the function's local environment.

The built-in `print()` function uses exactly the same surface syntax as other calls:

```
1 print(x);
2 print(3.14);
```

Return type uniformity. A function’s return type is inferred from its return statements. All return expressions within a function body must yield the same type; mixed return types are a static type error. A function with no return statement is assigned the special internal type `Void` and may only be invoked as a standalone statement, not used as an expression. Assigning the result of a `Void` function to a variable (e.g. `x = f()`; where `f` returns nothing) is rejected by the type checker. Passing a `Void`-returning function as an argument — for example `print(f())` where `f` returns nothing — is also rejected, since `Void` is not a printable value.

2.6 Operator Precedence and Associativity

Operator precedence is encoded structurally in the recursive-descent parser. Comparison is lowest, addition and subtraction are next, and multiplication and division have the highest precedence among binary operators. Arithmetic operators are left-associative.

Table 2.2. Operator Precedence (1 = lowest, 3 = highest) and Associativity

Precedence	Operators	Associativity
1 (lowest)	<code>==, !=</code>	non-associative (one comparison per expression)
2	<code>+, -, +., -.</code>	left-associative
3 (highest)	<code>*, /, *., /. </code>	left-associative

Table 2.3. Formal arithmetic and comparison operator signatures

Operator	Operand types	Result type
<code>+</code>	<code>Integer × Integer</code>	<code>Integer</code>
<code>-</code>	<code>Integer × Integer</code>	<code>Integer</code>
<code>*</code>	<code>Integer × Integer</code>	<code>Integer</code>
<code>/</code>	<code>Integer × Integer</code>	<code>Integer</code>
<code>+. </code>	<code>Float × Float</code>	<code>Float</code>
<code>-. </code>	<code>Float × Float</code>	<code>Float</code>
<code>*. </code>	<code>Float × Float</code>	<code>Float</code>
<code>/. </code>	<code>Float × Float</code>	<code>Float</code>
<code>==</code>	<code>Integer × Integer</code>	<code>Boolean</code>
<code>==</code>	<code>Float × Float</code>	<code>Boolean</code>
<code>!=</code>	<code>Integer × Integer</code>	<code>Boolean</code>
<code>!=</code>	<code>Float × Float</code>	<code>Boolean</code>

Any operand combination not listed in Table ?? is rejected during static type checking. Each operator token maps to exactly one valid operand signature (or, for `==` and `!=`, two same-type signatures), operator meaning can be determined without overload resolution or implicit type coercion.

2.7 Design Decisions: No Overloading

The language definition adopted for this project states: “No overloading with the usual precedence of multiplication and division over addition and subtraction. Hence no explicit declaration.” This section explains how the language design satisfies this requirement and why alternatives were rejected.

Definition of overloading. An operator is overloaded when the same syntactic symbol carries more than one type signature, requiring the compiler to inspect operand types to determine the operator’s meaning or result type. Under this definition, a language where $+$ means both integer addition $(Integer, Integer) \rightarrow Integer$ and floating-point addition $(Float, Float) \rightarrow Float$ — or worse, mixed-type addition $(Integer, Float) \rightarrow Float$ — overloads the $+$ symbol.

Distinct operator tokens. Following the Caml Light convention, this language assigns a unique token to each arithmetic meaning. Integer arithmetic uses $+$, $-$, $*$, $/$; floating-point arithmetic uses $+. , - . , * . , / .$. Each token has exactly one type signature. The type of any expression can be determined from the operator token alone, without inspecting operand types. This implementation adopts a strict interpretation of the specification’s “no overloading” requirement.

Rejected alternative: implicit numeric promotion. An alternative design would use a single set of operators and silently promote Integer operands to Float when mixed with a Float operand (e.g. $1 + 2.5 \rightarrow 3.5 : Float$). Proponents of this approach argue that promotion is not overloading because the operator retains a single “arithmetic addition” meaning. However, this argument is difficult to sustain formally: the operator $+$ would have three distinct input-type combinations — $(Integer, Integer)$, $(Integer, Float)$, and $(Float, Float)$ — with two different result types, a dispatch table that the compiler must resolve from operand types, which is the hallmark of overloading. The same language definition also states “hence no explicit declaration,” implying that type inference should be unambiguous; implicit promotion introduces cases where the inferred result type depends on operand types rather than on the operator itself.

Connection to type inference. Since every operator token uniquely determines its operand and result types, the type inference algorithm described in Chapter 5 can determine the type of any expression without backtracking, global unification, or global constraint propagation. This determinism directly satisfies the specification’s coupling of “no overloading” with “no explicit declaration.”

Table ?? contrasts the two designs.

Table 2.4. Comparison of operator design approaches

Expression	This language (separate tokens)	Promotion-based language
<code>3 + 4</code>	<code>7 : Integer (+ → Integer)</code>	<code>7 : Integer</code>
<code>3.0 +. 4.0</code>	<code>7.0 : Float (+. → Float)</code>	N/A (no <code>+. </code> operator)
<code>3 + 4.0</code>	<code>TypeError: + requires two Integer operands</code>	<code>7.0 : Float</code> (implicit promotion)
<code>10 / 3</code>	<code>3 : Integer</code> (floor division)	<code>3.333... : Float</code> (or <code>3 : Integer</code> , design-dependent)
Type signatures of <code>+</code>	One: <code>(Integer,Integer) → Integer</code>	Three: <code>(Integer,Integer)</code> , <code>(Integer,Float)</code> , <code>(Float,Float)</code>
Overloaded?	No	Arguably yes

CHAPTER 3

LEXICAL ANALYSIS AND SYMBOL TABLE

This chapter covers the responsibilities related to lexical and early semantic foundations of the language. It includes defining token structures and categories (such as keywords, operators, identifiers, and literals), implementing the lexer to transform source characters into tokens, and establishing the symbol table to manage variable and function storage along with their associated types. Together, these components ensure that raw source input is systematically organised into meaningful structures that can be used reliably by later stages of the compiler.

3.1 Token representation

The token system, written in `components/tokens.py`, establishes the vocabulary and structure of all lexical elements in the language. It specifies both the types of tokens that can exist and the structure each token must follow. This serves as the canonical interface between lexical analysis and syntax analysis, ensuring consistency across all stages of the compiler.

Table ?? summarises the main categories, example lexemes, and typical `TokenType` values.

Table 3.1. Token categories and representative `TokenType` values

Category	Example lexemes	Representative <code>TokenType</code>
Literals	42, 3.14, true, "hello"	INT_LITERAL, FLOAT_LITERAL, BOOL_LITERAL, STRING_LITERAL
Identifiers	x, counter, add	IDENTIFIER
Keywords	if, else, while, def, return	keyword-specific token variants
Integer operators	+, -, *, /	PLUS, MINUS, STAR, SLASH
Float operators	+, -, *, /	PLUS_DOT, MINUS_DOT, STAR_DOT, SLASH_DOT
Other operators	=, ==, !=	ASSIGN, EQUAL_EQUAL, BANG_EQUAL
Delimiters	Parentheses, braces, comma, and semicolon (see lexer delimiter tokens)	delimiter token variants
End marker	end of source input	EOF

Token types (from class `TokenType`) are defined using an Enum with `auto()` to automatically assign unique values. This avoids reliance on raw strings and ensures safer comparisons during parsing. The token categories include literals, identifiers, keywords, operators, delimiters,

and a special end-of-file (EOF) token. The EOF token is appended after all input is processed, allowing the parser to detect the end of input explicitly rather than encountering an unexpected termination.

Each token is represented as an immutable dataclass (`frozen=True`), meaning its fields cannot be modified after creation. Each token contains four key pieces of information: its kind (`token_type`), its lexeme (the exact substring from the source code), and its source position (`line`, `column`). This structured representation allows later stages to process the program in terms of syntax and errors without handling raw text directly. Immutability improves reliability by preventing accidental modification during later processing.

```
1 @dataclass(frozen=True)
2 class Token:
3     token_type: TokenType
4     lexeme: str
5     line: int
6     column: int
```

3.2 Lexer implementation

The lexer, implemented in `components/lexica.py`, is a hand-written scanner that converts raw source text into a sequence of tokens. Unlike generated lexers (for example SLY lexer mode, or Lex and Flex), this implementation uses explicit control flow (`if` and `while`) to process input.

The lexer maintains internal state including the source string, current position, start of the current token, line number, and column index. The column index is tracked per line and resets upon encountering a newline, while a separate `token_column` records the starting position of each token for accurate error reporting.

The main function, `tokenize`, iterates through the source code, repeatedly invoking `_scan_token` to identify the next token. Tokens are appended to a list, and an EOF token is added at the end. Whitespace is skipped, while invalid input results in a `LexerError`.

```
1 def tokenize(self) -> list[Token]:
2     tokens: list[Token] = []
3     while not self._is_at_end():
4         self.start = self.current
5         self.token_column = self.column
6         token = self._scan_token()
7         if token is not None:
8             tokens.append(token)
9     tokens.append(Token(TokenType.EOF, "", self.line, self.column))
```

```
10 return tokens
```

Token recognition follows a structured approach. Single-character tokens are mapped using a lookup table. Multi-character operators such as `==` and `!=` are handled using lookahead via `_match`. String literals are processed by `_string`, which reads characters until a closing quotation mark or the end of input is reached, with error handling for unterminated strings. Numeric literals are handled by `_number`, which distinguishes integers and floating-point values based on the presence of a decimal point followed by digits. Identifiers are processed greedily, consuming alphanumeric characters and underscores, and then checked against the `KEYWORDS` map to determine whether they are reserved words or user-defined names. Notably, `true` and `false` are also entries in this map, mapping to `TokenType.BOOL_LITERAL` rather than a keyword-specific type. Boolean literals are therefore recognised through the same keyword-fallback mechanism as reserved words, preventing them from being scanned as user-defined identifiers. In contrast, `print` is *not* reserved: it is scanned as an ordinary `IDENTIFIER` and later treated as a built-in callable by the type checker and interpreter.

```
1 KEYWORDS: dict[str, TokenType] = {
2     "if": TokenType.IF,
3     "else": TokenType.ELSE,
4     "while": TokenType.WHILE,
5     # ...
6     "true": TokenType.BOOL_LITERAL,
7     "false": TokenType.BOOL_LITERAL,
8 }
```

```
1 # Arithmetic operators with optional trailing '.' for float variant
2 _ARITH_DOT: dict[str, tuple[TokenType, TokenType]] = {
3     "+": (TokenType.PLUS, TokenType.PLUS_DOT),
4     "-": (TokenType.MINUS, TokenType.MINUS_DOT),
5     "*": (TokenType.STAR, TokenType.STAR_DOT),
6     "/": (TokenType.SLASH, TokenType.SLASH_DOT),
7 }
```

When one of these characters is encountered, the lexer checks the next character: if it is a `.` (and the character after that is not a digit), the dot-suffixed float variant is emitted; otherwise the plain integer operator is emitted. This lookahead prevents the `.` in a float literal such as `3.14` from being consumed as part of an operator.

```
1 SINGLE_CHAR_TOKENS: dict[str, TokenType] = {
2     "=": TokenType.ASSIGN,
3     "(": TokenType.LPAREN,
4     ")": TokenType.RPAREN,
5     # ...
```

Helper functions such as `_advance`, `_peek`, and `_peek_next` manage character consumption and lookahead, while `_is_at_end` determines when input has been fully processed. Error messages include precise line and column information, aiding debugging and diagnostics.

Table ?? lists the main functions and data structures in `components/lexica.py` and their roles.

Table 3.2. Key functions and data in `lexica.py`

Function / data	Responsibility	Output / effect
<code>tokenize()</code>	Iterates through source, scans tokens, appends EOF	ordered token list
<code>_scan_token()</code>	Dispatches recognition based on character and lookahead	next token or skip
<code>_number()</code>	Parses numeric sequences, distinguishes int vs float	numeric literal token
<code>_string()</code>	Parses quoted text, checks termination	string token or error
<code>_identifier()</code>	Parses identifiers, resolves keyword vs user-defined	keyword or identifier token
KEYWORDS map	Maps reserved words and Boolean literals to token types	token type selection
SINGLE_CHAR_TOKENS map	Maps single-character symbols	token type selection
<code>_advance()</code> , <code>_peek()</code> , <code>_match()</code>	Character consumption and lookahead	scanner state update

3.3 Symbol table and semantic structure

The symbol table, defined in `components/symbol_table.py`, operates at a higher level than the lexer. While the lexer processes raw text into tokens, the symbol table manages semantic information such as variable and function names, their types, and their scope.

The symbol table is implemented as a scoped structure, where each scope maintains its own dictionary of symbols and optionally references a parent scope. This enables proper handling of nested blocks and variable shadowing. Symbols are introduced through definition functions and retrieved through lookup operations, which search recursively through parent scopes if needed.

```

1 def lookup(self, name: str) -> Symbol:
2     symbol = self._symbols.get(name)
3     if symbol is not None:
4         return symbol
5     if self.parent is not None:

```

```

6         return self.parent.lookup(name)
7         raise SymbolTableError(f"Undefined symbol: {name}")

```

The type system is represented using a `LanguageType` enumeration, which includes fundamental types such as `Integer`, `Float`, `Boolean`, `String`, and `Void`. Symbols are modelled using an inheritance hierarchy: the base `Symbol` class stores a name and type, while subclasses such as `VariableSymbol` and `FunctionSymbol` include additional attributes. Variables track initialization state, while functions store parameter lists and return types. Table ?? summarises the main building blocks.

Table 3.3. Symbol table components and roles

Component	Stored information	Role in semantic analysis
Scoped symbol table	Symbols in current and parent scope	Supports nesting and shadowing via recursive lookup
<code>LanguageType</code> enum	<code>Integer</code> , <code>Float</code> , <code>Boolean</code> , <code>String</code> , <code>Void</code>	Defines the type system
<code>Symbol</code> (base)	Name and declared or inferred type	Base abstraction for entries
<code>VariableSymbol</code>	Variable metadata (including initialization state)	Tracks correct usage before assignment
<code>FunctionSymbol</code>	Parameters and return type	Validates calls and return consistency
<code>lookup(name)</code>	Recursive scope search	Resolves identifiers or reports undefined
<code>SymbolTableError</code>	Semantic error details	Reports duplicates, undefined symbols, invalid operations

Errors related to semantic violations are handled via `SymbolTableError`. These include duplicate declarations within the same scope, references to undefined symbols, and invalid operations such as attempting to mark a non-variable symbol as initialized.

3.3.1 Example symbol table hierarchy

Figure ?? illustrates a small two-scope symbol table after the type checker has processed a program containing two top-level variables and a function definition.

Global scope		
Name	Kind	Type
x	Variable	Integer
y	Variable	Float
addf	Function	Float $\times$ Float $\rightarrow$ Float
Function scope: addf		
Name	Kind	Type
a	Parameter	Float
b	Parameter	Float

Figure 3.1. Example symbol table hierarchy with one global scope and one function scope.

The global scope stores top-level variables and function definitions, while each function invocation creates a child scope containing parameter bindings and local variables. Lookup operations recursively search parent scopes when identifiers are not found in the current environment.

3.4 Role of the symbol table in the pipeline

The symbol table is constructed during the type-checking phase rather than during lexical analysis. Accordingly, the type checker initialises a global symbol table and populates it while traversing the AST, defining variables and functions, managing nested scopes, and validating type correctness.

```

1 def run_pipeline(source_code: str) -> PipelineResult:
2     tokens = Lexer(source_code).tokenize()
3     tree = Parser(tokens).parse()
4     parse_tree_text = ParseTreePrinter().print(tree)
5     ast_text = ASTPrinter().print(tree)
6     checked_scope = TypeChecker().check(tree)
7     outputs: list[str] = []
8     Interpreter(output_callback=outputs.append).execute(tree)
9     return PipelineResult(
10         tokens=tokens,
11         parse_tree_text=parse_tree_text,
12         ast_text=ast_text,
13         checked_scope=checked_scope,
14         outputs=outputs,
15     )

```

Once type checking is complete, the populated symbol table is returned as part of the pipeline result (`checked_scope`). The checker creates child scopes for blocks and functions during analysis, but the displayed `checked_scope` table represents the global scope entries returned

by the pipeline. It can then be displayed or used for further analysis.

This separation of concerns keeps lexical analysis focused on tokenization, while semantic meaning is enforced during type checking. For user-defined functions, definitions are pre-registered in the global scope so that forward references can be resolved during checking. For functions that are called, parameter types are fixed from the first call site encountered and held constant; the function body is then verified with those fixed types. All subsequent calls must supply arguments that match the established types. For functions that are never called, a separate post-pass infers types from body usage.

CHAPTER 4

RECURSIVE-DESCENT PARSING, PARSE TREE, AND ABSTRACT SYNTAX TREE

This chapter presents the implementation of the recursive-descent parser, the grammar-oriented parse-tree view exported for inspection, and the associated abstract syntax tree (AST) model used for downstream semantic analysis and execution. Starting from lexer-produced tokens, the parser constructs a hierarchical Program AST. A separate `ParseTreePrinter` then renders the parsed program in an ASCII branch-style tree. This makes the correspondence between grammar productions and accepted programs explicit without changing the executable internal representation. Implemented syntax coverage includes arithmetic expressions, assignments, if-then-else, while, function definitions, function calls, return, and built-in `print(...)` calls. Equality operators (`==`, `!=`) are parsed as ordinary lowest-precedence expressions, so Boolean results can appear in conditions, assignments, function arguments, and `print(...)` expressions.

4.1 Abstract syntax tree representation

The AST is defined as a collection of Python dataclasses representing the structural elements of the programming language. It serves as an intermediate representation between parsing and later stages such as type checking and interpretation.

A key distinction exists between tokens and AST nodes. Tokens are flat and describe individual lexical elements such as operators or identifiers. In contrast, the AST is hierarchical. For example, while a token may represent the symbol `+`, a `BinaryOp` node represents an entire addition operation, including its left and right operands as subtrees.

All AST nodes inherit from a common base class, allowing them to store positional metadata (`line`, `column`). This ensures that errors detected in later stages can still reference precise locations in the original source code.

```
1 @dataclass
2 class ASTNode:
3     """Base class for every AST node. Carries source location for diagnostics."""
4     line: int = field(default=0, kw_only=True)
5     column: int = field(default=0, kw_only=True)
```

4.1.1 Expressions

Expressions represent computations or values within the program. A `Literal` node stores values known at parse time, including integers, floating-point numbers, booleans, and strings. String literals are stored without surrounding quotation marks, as these are removed during parsing.

```
1 if tok.token_type == TokenType.STRING_LITERAL:
2     self._advance()
3     return Literal(value=tok.lexeme[1:-1], line=tok.line, column=tok.column)
```

An `Identifier` node represents the use of a variable name within an expression, indicating evaluation rather than declaration. A `BinaryOp` node models arithmetic and comparison operations such as addition, subtraction, multiplication, division, and equality checks, storing the operator along with references to left and right operands. A `FunctionCall` node represents function invocation, consisting of the function name and a list of argument expressions.

```
1 @dataclass
2 class BinaryOp(ASTNode):
3     """An arithmetic or comparison binary operation.
4
5     op is one of: '+', '-', '*', '/', '+.', '-.', '*.', '/.', '==', '!='
6     Example: a + b x == 0 a +. b
7     """
8     op: str
9     left: ASTNode
10    right: ASTNode
```

4.1.2 Statements

Statements represent actions or control flow within the program. An `Assign` node models variable assignment, pairing a variable name with a value expression, while `Return` stores a single expression and represents function return. A `FunctionCall` node may also appear as a standalone statement, which is how built-in `print(...)` calls are represented after parsing. A `Block` node groups multiple statements within braces, representing a scoped execution sequence.

Control flow is handled via `If` and `While` nodes. The `If` node stores a condition, a then-block, and an optional else-block, while `While` stores a loop condition and body block. Function definitions are represented by `FunctionDef` nodes, which include the function name, a list of

parameter names, and a body block. Parameter types are intentionally not assigned at parse time, as type inference is handled later by the type checker. Finally, `Program` serves as the AST root, containing all top-level statements.

4.2 Parser implementation

The parser consumes the list of tokens generated by the lexer and produces a single `Program` AST node. It is implemented as a hand-written recursive-descent parser based on the project grammar. This avoids parser generators and instead relies on explicit functions for each grammar rule.

The parser maintains internal state consisting of the token list and a position index (`self._pos`) indicating the current token. Tokens are accessed via helper methods such as `_peek` and `_advance`. Parsing errors are reported using a custom `ParseError` exception with line and column information consistent with lexer diagnostics.

```
1 def parse(self) -> Program:
2     """Parse the full token stream and return the root Program node."""
3     line, col = self._peek().line, self._peek().column
4     statements: list[ASTNode] = []
5     while not self._is_at_end():
6         statements.append(self._statement())
7     return Program(statements=statements, line=line, column=col)
```

In addition to the main `parse()` entry point, the parser's statement dispatcher (`_statement`) implements a deterministic decision order over token patterns: function definition, conditional, loop, return, assignment, then expression statement fallback. This design is important because it prevents ambiguity between syntactically similar forms. In particular, assignments are recognized using one-token lookahead (`IDENTIFIER` followed by `=`), while identifier-led expressions that are not assignments are parsed as expression statements and must still end with a semicolon. This gives the parser full coverage of both declarative-style and expression-driven top-level statements without requiring a parser generator.

4.2.1 Rationale for recursive-descent parsing

Recursive-descent parsing was selected because the project grammar is sufficiently structured to permit predictive parsing with explicit precedence layering. This approach avoids parser generators and keeps parsing logic directly aligned with grammar productions, making the implementation easier to debug, extend, and explain.

Operator precedence is encoded structurally through layered parsing methods (`_expr`, `_additive`, `_term`, `_factor`), eliminating the need for runtime precedence tables. The hand-written approach also provides explicit control over lookahead and ambiguity resolution, particularly for distinguishing assignments from identifier-led expression statements.

4.3 Parse-tree view

Although the parser’s internal product is an AST, the system also exposes a separate parse-tree view for reporting and demonstration. The renderer is implemented in `components/parse_tree_printer.py` and operates on the parsed program using grammar-oriented layers such as `program`, `statement`, `assignment`, `expr`, `comparison`, `additive`, `term`, `factor`, `function_call`, and `args`. The `comparison` layer wraps the operands of `==` and `!=` when they appear at the top of an expression. Terminal-like leaves are rendered as `IDENTIFIER(x)`, `INT_LITERAL(5)`, `'(', ' ', ';'`, and `EOF`, with ASCII branch markers indicating parent-child structure.

This design keeps the semantic implementation simple: the type checker and interpreter still consume the AST, while the parse-tree renderer exists purely to expose the grammar structure required for inspection.

Table 4.1. Parse tree versus AST in the implemented system

Artifact	Purpose	Used by
Parse tree	Grammar-oriented inspection view	CLI, UI, report
AST	Executable semantic tree	Type checker, interpreter

Figure ?? illustrates the source fragment `a = 3;`. The report uses this compact diagram for explanation, whereas the implemented system produces the scalable ASCII branch representation shown in the CLI and UI.

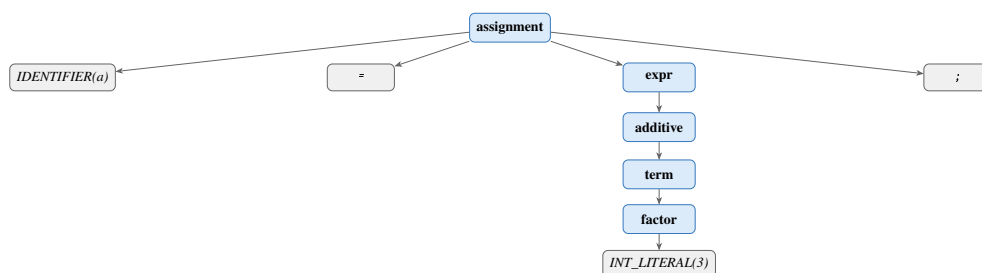


Figure 4.1. Parse-tree diagram for `a = 3;`.

4.3.1 Statement parsing

The main parsing loop repeatedly processes statements until EOF, with the statement type determined via dispatch on the current token. Function definitions, conditionals, loops, and returns each have dedicated parsing methods. Assignments are detected using one-token lookahead (IDENTIFIER followed by ASSIGN), preventing misclassification of function calls as assignments. If no specific statement pattern matches, the parser falls back to an expression statement and enforces a terminating semicolon.

```
1 # assignment: IDENTIFIER ASSIGN expr SEMICOLON
2 if (
3     tok.token_type == TokenType.IDENTIFIER
4     and self._peek_next().token_type == TokenType.ASSIGN
5 ):
6     return self._assignment()
7 # fallback: bare expression statement expr SEMICOLON
8 node = self._expr()
9 self._expect(TokenType.SEMICOLON, "Expected ';' after expression")
```

4.3.2 Expression parsing and precedence

Expressions are parsed in layered precedence levels. `_expr` handles comparison operators (`==`, `!=`) after parsing an additive left-hand side. `_additive` then handles `+`, `-`, `+. .`, and `-. .`; `_term` handles `*`, `/`, `*. .`, and `/. .`; and `_factor` handles literals, identifiers, function calls, and parenthesized expressions. Function calls are recognised when an identifier is followed by `(`, with argument lists parsed as comma-separated expressions. Parenthesized expressions override precedence explicitly. Unary negation (`-x`, `-5`) is also handled at the `_factor` level: a leading `-` token causes the parser to recursively parse the operand and return `BinaryOp(0 - operand)`, allowing the existing subtraction rules to cover negation without additional inference logic. Float unary negation is handled similarly for `-. .`, producing `BinaryOp(0.0 -. operand)`.

```
1 def _expr(self) -> ASTNode: # ==, !=
2     ...
3 def _additive(self) -> ASTNode: # +, -, +. ., -. .
4     ...
5 def _term(self) -> ASTNode: # *, /, *. ., /. .
6     ...
7 def _factor(self) -> ASTNode: # unary -, literals, identifiers/calls, grouped expr
8     ...
```

Figure ?? shows the AST produced for the statement `z = x + y * 2;`. As `_term` handles `*`

before `_additive` handles `+`, the multiplication sub-tree is nested one level deeper, encoding the higher precedence of `*` over `+` without any explicit priority table at runtime.

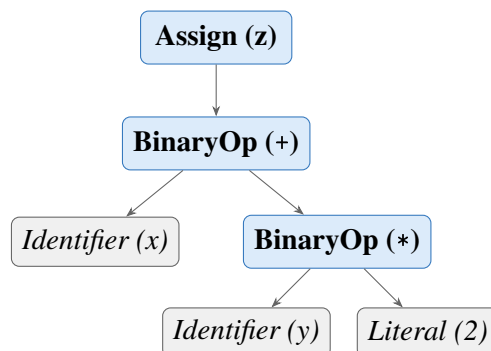


Figure 4.2. AST for `z = x + y * 2;`. Multiplication is a deeper sub-tree than addition, encoding higher precedence through tree structure rather than a runtime priority table.

This structural encoding of precedence eliminates the need for runtime operator-priority resolution and keeps semantic analysis independent of parsing strategy.

Expression parsing is intentionally split into layered methods to enforce precedence and associativity in a transparent way. Parenthesized expressions are parsed explicitly and re-enter `_expr`, allowing precedence override where required by source syntax. Comparison operators (`==`, `!=`) are parsed at the top of the `_expr` method as the lowest-precedence binary operators. As comparisons are part of the general expression grammar, Boolean results can appear in any expression context — as conditions, assignments, arguments, or print expressions. Semantic enforcement that a condition is truly Boolean is deferred to the type checker.

Figure ?? shows the AST for an `if` statement with a comparison condition, illustrating how the parser structures control flow with a Boolean sub-tree.

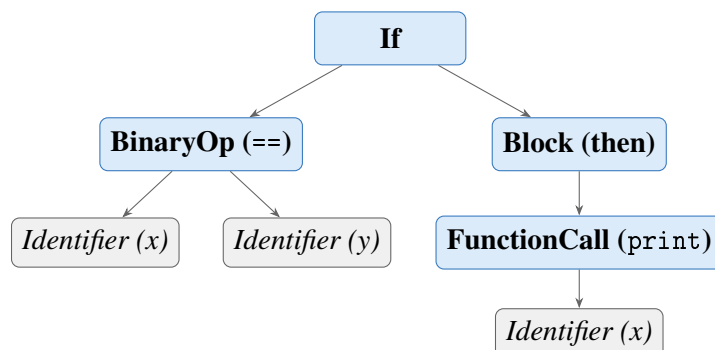


Figure 4.3. AST for `if (x == y) { print(x); }`. The comparison produces a **BinaryOp** node whose inferred type is **Boolean**, checked by the type checker before execution.

4.3.3 *Helper functions and lookahead*

As described above, the parser uses `_check`, `_expect`, `_peek`, `_peek_next`, `_advance`, and `_is_at_end` for token navigation and validation. Lookahead is central for ambiguity resolution: assignment versus function call is resolved by inspecting the next token, while comparison operators are recognised in `_expr` after parsing the left additive expression. Parser errors follow a consistent diagnostic format: `[line X, col Y] ParseError: <message>`.

The parser also includes dedicated list parsers for formal parameters (`_params`) and call arguments (`_args`), both implemented as comma-separated sequences with optional emptiness. This ensures consistent handling of signatures such as `def f()` and `def f(a, b)` as well as invocations like `f()` and `f(x, y+1)`.

4.4 AST printing

For debugging and reporting, an AST printer converts the tree into a readable indented string, integrated into the pipeline output. The printer uses visitor-style dynamic dispatch, mapping node class names to methods such as `_visit_If` and `_visit_BinaryOp`, with output built as multi-line text where indentation reflects tree depth. Node formatting emphasises structure: `Program` and `Block` include statement counts, `BinaryOp` shows left and right subtrees, and `FunctionCall` prints each argument by index. `Literal` includes both value and runtime type names.

4.5 Parser and type checker boundary

A deliberate architectural boundary is maintained between syntax construction and semantic validation. The parser is responsible for producing a structurally valid AST with accurate source locations. The parse-tree renderer exposes the same parsed program in grammar-facing form, but it is not used for execution. Type correctness and semantic constraints are checked later by the type checker. For example, `_expr` may syntactically produce a node whose eventual type is not `Boolean`, but rejection of non-`Boolean` control-flow conditions is deferred to semantic analysis. This keeps parser logic focused on grammar conformance and preserves a clear responsibility split across team member contributions.

CHAPTER 5

TYPE INFERENCE ALGORITHM

5.1 Overview

Type inference in this project means that the programmer does not write explicit type declarations for variables or functions. Instead, the compiler infers types from assignments, literals, operations, function calls, and return statements. This keeps the language simple while still preserving static type safety. Specifically, the algorithm is *syntax-directed* and local: each expression's type is determined by its subexpressions and operator token alone, without unification variables or constraint solving.

The type checker runs after parsing. It receives the AST and validates the program before execution. Any inconsistent use of types is reported as a type error, and execution is stopped.

This implementation is not Hindley–Milner inference; it uses a syntax-directed local strategy with first-call parameter fixation for called functions and a body-usage post-pass for never-called functions.

The interpreter stage does not repeat static type validation: it walks the same AST with ordinary Python representations of values. Static type information remains in the symbol table produced by the checker; run-time values are not separately labelled with `LanguageType`, except that the built-in `print` formats output by inspecting the value's Python type.

5.2 Type Environment

The type environment is represented by the symbol table. Conceptually, it is a mapping

$$\Gamma : Identifier \rightarrow Type$$

The environment grows as the checker visits the AST, and on initial binding a variable name is inserted with its inferred type. When a new block or a function call is entered, a child environment is created so that nested scopes can be checked independently while still accessing outer bindings when needed.

5.3 Type Inference Procedure

The type checker performs type inference by traversing the abstract syntax tree before interpretation. The algorithm is syntax-directed: each AST node is visited, the types of its child nodes are inferred first, and the current node is checked according to the language typing rules.

Algorithm TYPECHECK(*node*, Γ)

Input: *node* = current AST node; Γ = current type environment / symbol table

Output: inferred type of *node*, or TypeError

1. **If** *node* is a **Literal**: return the corresponding primitive type: Integer, Float, Boolean, or String.
2. **If** *node* is an **Identifier**: look up the identifier in Γ . If it is not found, report an undefined variable error. Otherwise, return its stored type.
3. **If** *node* is a **BinaryOp**:
 - Infer the type of the left operand and the right operand.
 - If the operator is +, -, *, or /: require both operands to be Integer; return Integer.
 - If the operator is +., -., *., or /.: require both operands to be Float; return Float.
 - If the operator is == or !=: require both operands to be the same arithmetic type; return Boolean.
 - Otherwise, report a TypeError.
4. **If** *node* is an **Assignment**:
 - Infer the type of the right-hand-side expression.
 - If the variable is not yet in Γ : insert it with the inferred type.
 - Else: verify the new type matches the existing type; if not, report a TypeError.
5. **If** *node* is an **If** statement:
 - Infer the condition type; require it to be Boolean.
 - Type-check the then-block in a child environment.
 - If an else-block exists, type-check it in a child environment.
6. **If** *node* is a **While** statement:

- Infer the condition type; require it to be `Boolean`.
- Type-check the loop body in a child environment.

7. If *node* is a **FunctionDef**:

- Register the function name in Γ .
- Parameter types start as placeholders; for called functions they are fixed by the first call site, while never-called functions are assigned using body-usage inference in the post-pass.
- Return type is inferred from `return` statements in the body.

8. If *node* is a **FunctionCall**:

- If the function is the built-in `print`: require exactly one argument, infer that argument type, reject `Void` arguments, and return `Void`.
- Otherwise, infer all argument types, validate them against the function signature, type-check the callee body if needed, and return the inferred function return type.

5.3.1 Example type inference trace

Table ?? traces the algorithm step by step for the program fragment

```

1 x = 3;
2 y = 4;
3 z = x + y;

```

showing how the type environment Γ grows as each sub-expression is inferred.

Table 5.1. Step-by-step type inference trace for `x = 3; y = 4; z = x + y;`.

Step	Expression	Inferred type	Environment Γ
1	3	Integer	$\emptyset$
2	x = 3	Integer	$\{x: \text{Integer}\}$
3	4	Integer	$\{x: \text{Integer}\}$
4	y = 4	Integer	$\{x: \text{Integer}, y: \text{Integer}\}$
5	x + y	Integer	unchanged
6	z = x + y	Integer	$\{x: \text{Integer}, y: \text{Integer}, z: \text{Integer}\}$

5.4 Inference Rules

5.4.1 Judgement Forms

The static semantics are written using the following judgement forms:

$$\Gamma \vdash e : T$$

which reads “under type environment Γ , expression e has type T ”, and

$$\Gamma \vdash e : \text{TypeError}$$

which denotes a statically rejected expression.

5.4.2 Literals

Literal nodes have direct types:

$$\begin{array}{ll} \frac{}{\Gamma \vdash 42 : \text{Integer}} \text{ (T-INTLIT)} & \frac{}{\Gamma \vdash 3.14 : \text{Float}} \text{ (T-FLOATLIT)} \\ \frac{}{\Gamma \vdash \text{true} : \text{Boolean}} \text{ (T-BOOLLIT)} & \frac{}{\Gamma \vdash \text{"hi"} : \text{String}} \text{ (T-STRINGLIT)} \end{array}$$

5.4.3 No Operator Overloading

The language does not support operator overloading. Following the Caml Light convention, integer and float arithmetic use entirely separate operator tokens, so every operator has a single, fixed type signature:

- $+$, $-$, $*$, $/$ — require two `Integer` operands; result is `Integer`.
- $+. , -. , *. , /. ,$ — require two `Float` operands; result is `Float`.
- $+$ is not overloaded for string concatenation; applying it to a `String` is a type error.
- $==$ and $!=$ are not overloaded for `Boolean` or `String` equality.
- No implicit numeric promotion: `Integer + Float` is a type error.

As every operator has a unique, unambiguous type derivable from its token alone, the compiler can always infer expression types without requiring explicit type declarations from the

programmer. This design ensures that operator meaning is determined uniquely from the operator token itself, eliminating semantic ambiguity during type inference and simplifying static analysis.

5.4.4 Arithmetic Expressions

The integer operators $+$, $-$, $*$, and $/$ require both operands to be `Integer` and always return `Integer` (including division, which uses integer floor division). The float operators $+. , - . , * . , / .$ require both operands to be `Float` and always return `Float`. Mixing an `Integer` and a `Float` with any operator is a type error.

Table 5.2. Arithmetic operator type rules.

Left operand	Operator	Right operand	Result type
Integer	$+, -, *, /$	Integer	Integer
Float	$+. , - . , * . , / .$	Float	Float
Integer	any	Float	TypeError
Float	any	Integer	TypeError
Any	any	String or Boolean	TypeError

The following grouped derivation rules capture these constraints:

$$\frac{\Gamma \vdash e_1 : \text{Integer} \quad \Gamma \vdash e_2 : \text{Integer}}{\Gamma \vdash e_1 \text{ op } e_2 : \text{Integer}} \text{ (T-INTOP), op} \in \{+, -, *, /\}$$

$$\frac{\Gamma \vdash e_1 : \text{Float} \quad \Gamma \vdash e_2 : \text{Float}}{\Gamma \vdash e_1 \text{ op } e_2 : \text{Float}} \text{ (T-FLOATOP), op} \in \{+. , - . , * . , / .\}$$

$$\frac{\Gamma \vdash e_1 : \text{Integer} \quad \Gamma \vdash e_2 : \text{Float}}{\Gamma \vdash e_1 \text{ op } e_2 : \text{TypeError}} \text{ (T-MIXEDERROR)}$$

For the concrete mixed arithmetic expression discussed in this report:

$$\Gamma \vdash 3 + 4.0 : \text{TypeError}$$

Note that each rule is keyed on the operator token, not on the operand types. This is only possible because integer and float arithmetic use distinct tokens — the property that eliminates operator overloading.

Unary negation

Unary integer negation ($-x$) is represented internally as the equivalent subtraction form $0 - x$. Unary float negation ($-.\ x$) is represented internally as $0.0 -.\ x$. The type checker therefore applies the standard binary rules in both cases.

5.4.5 Boolean Expressions

The comparison operators `==` and `!=` always produce `Boolean`. Both operators require two operands of the *same* arithmetic type — either both `Integer` or both `Float`; mixed-type comparisons such as `Integer == Float` are rejected. Comparisons involving `Boolean` or `String` operands are also rejected by the type checker. Although `Boolean` and `String` values exist in the language, equality and inequality are deliberately restricted to arithmetic expressions because the project specification defines basic `Boolean` expressions as equality and inequality between two arithmetic expressions.

Furthermore, the same-type requirement for comparisons — both operands must be either both `Integer` or both `Float` — is a direct consequence of the no-overloading design. If `==` accepted mixed `Integer` and `Float` operands, the compiler would need to determine whether to compare by promotion (implicitly converting `Integer` to `Float`) or by value identity, introducing the same ambiguity that the separate-operator design eliminates for arithmetic. Rejecting mixed-type comparisons ensures that `==` and `!=` each have exactly two valid type signatures — $(\text{Integer}, \text{Integer}) \rightarrow \text{Boolean}$ and $(\text{Float}, \text{Float}) \rightarrow \text{Boolean}$ — rather than a third mixed-type signature.

5.4.6 Assignment Statement

Assignments define variable types on first use. If a variable does not yet exist in the type environment, the checker infers the type of the right-hand side and inserts the variable into the environment. If the variable already exists, the new value must have the same type.

$$\frac{x \notin \Gamma \quad \Gamma \vdash e : T}{\Gamma \vdash x = e : (\Gamma, x : T)}$$
$$\frac{x : T \in \Gamma \quad \Gamma \vdash e : T}{\Gamma \vdash x = e : \Gamma}$$
$$\frac{x : T_1 \in \Gamma \quad \Gamma \vdash e : T_2 \quad T_1 \neq T_2}{\Gamma \vdash x = e : \text{TypeError}}$$

5.4.7 If-Then-Else

The condition of an `if` statement must be Boolean. The then-block and else-block are checked in child scopes so that local operations can be validated cleanly.

$$\frac{\Gamma \vdash c : \text{Boolean} \quad \Gamma' \vdash B_{then} \quad \Gamma'' \vdash B_{else}}{\Gamma \vdash \text{if}(c) B_{then} \text{ else } B_{else}}$$
$$\frac{\Gamma \vdash c : T \quad T \neq \text{Boolean}}{\Gamma \vdash \text{if}(c) B_{then} \text{ else } B_{else} : \text{TypeError}}$$

5.4.8 While-Loop

The condition of a `while` loop must also be Boolean. The loop body is checked in a child scope and must not violate the types of variables already introduced in the surrounding environment.

$$\frac{\Gamma \vdash c : \text{Boolean} \quad \Gamma' \vdash B}{\Gamma \vdash \text{while}(c) B}$$
$$\frac{\Gamma \vdash c : T \quad T \neq \text{Boolean}}{\Gamma \vdash \text{while}(c) B : \text{TypeError}}$$

5.4.9 Function Definition and Call

Function definitions are pre-registered in the global scope so that forward references can be resolved. For functions that are called, parameter types are fixed at the first call site encountered and then held constant for all subsequent calls, which must supply arguments of the same types. The return type is inferred from the `return` statements in the function body, which is verified with those fixed parameter types. Inconsistent return types are reported as a type error. Furthermore, if a function's inferred return type is non-`Void`, the type checker enforces that the function returns on every execution path; a body where `return` appears only inside one branch of an `if` without a corresponding `else` is rejected as a non-total function.

For functions that are defined but never called, the type checker performs a post-pass after all top-level statements have been processed. It infers parameter types by scanning the body for arithmetic and comparison usage, then checks the full body with those inferred types. This ensures that type errors inside never-called function bodies are still detected before execution.

Recursive calls are intentionally outside the scope of this implementation; supporting them would require additional fixed-point inference or explicit recursion handling that is orthogonal

to the core type-inference algorithm.

Formal rules for function inference

Let Σ denote the function-signature environment, where $\Sigma(f) = (T_1, \dots, T_n) \rightarrow T_r$.

First-call parameter binding for called functions:

$$\frac{\Gamma \vdash a_1 : T_1 \quad \dots \quad \Gamma \vdash a_n : T_n \quad f \notin \Sigma}{\Sigma \cup \{f : (T_1, \dots, T_n) \rightarrow \text{Void}\}, \Gamma \vdash f(a_1, \dots, a_n)} \text{ (F-FIRSTCALL)}$$

Subsequent call checking:

$$\frac{\Sigma(f) = (T_1, \dots, T_n) \rightarrow T_r \quad \Gamma \vdash a_1 : T_1 \quad \dots \quad \Gamma \vdash a_n : T_n}{\Sigma, \Gamma \vdash f(a_1, \dots, a_n) : T_r} \text{ (F-CALL)}$$

Return-type consistency:

$$\frac{\Sigma(f) = (T_1, \dots, T_n) \rightarrow \text{Void} \quad \Gamma_f \vdash r_1 : T \quad \dots \quad \Gamma_f \vdash r_k : T}{\Sigma(f) = (T_1, \dots, T_n) \rightarrow T} \text{ (F-RETURNCONSISTENT)}$$

$$\frac{\Gamma_f \vdash r_i : T_i \quad \Gamma_f \vdash r_j : T_j \quad T_i \neq T_j}{\Gamma_f \vdash \text{FunctionDef}(f) : \text{TypeError}} \text{ (F-RETURNMISMATCH)}$$

Never-called function fallback used in this implementation:

$$\frac{p \in \text{params}(f) \quad p \text{ has no arithmetic-context constraint}}{\Sigma(f).p = \text{Integer}} \text{ (F-UNCALLEDDEFAULTINT)}$$

Worked derivation (single call):

```

1 def add(a, b) {
2   return a + b;
3 }
4
5 z = add(3, 4);
```

$$\Gamma \vdash 3 : \text{Integer} \quad \Gamma \vdash 4 : \text{Integer}$$

$$\Sigma(\text{add}) = (\text{Integer}, \text{Integer}) \rightarrow \text{Integer}$$

$$\frac{\Gamma \vdash 3 : \text{Integer} \quad \Gamma \vdash 4 : \text{Integer}}{\Sigma, \Gamma \vdash \text{add}(3, 4) : \text{Integer}}$$

Hence $\Gamma \vdash z = \text{add}(3, 4)$ extends Γ with $z : \text{Integer}$.

Value parameter passing example

The language uses value parameter passing, or call-by-value: each argument is evaluated in the caller's environment and a copy of its value is bound to the corresponding parameter in the callee's environment. Figure ?? shows the runtime environment before, during, and after a call to a function that mutates its parameter.

Using semantic-configuration notation with environment ρ and store σ , call-by-value is expressed as:

$$\begin{array}{c} \rho, \sigma \vdash e_i \Downarrow v_i, \sigma_i \quad (1 \leq i \leq n) \\ \rho_f = [x_1 \mapsto v_1, \dots, x_n \mapsto v_n] \\ \frac{\rho_f, \sigma_n \vdash B_f \Downarrow \sigma'}{\rho, \sigma \vdash f(e_1, \dots, e_n) \Downarrow v_{ret}, \sigma'} \quad (\text{S-CALLBYVALUE}) \end{array}$$

The key point is that each formal parameter x_i is bound to the evaluated argument value v_i in a fresh activation record for the callee.

The same behavior can be visualized as environment frames:

Table 5.3. Activation-record view for one call to f .

Record field	Content
Function name	f
Caller link	Global frame
Parameters	$a = 5$ (copied from caller argument x)
Locals after body step	$a = 6$
Return value	Void

Table 5.4. Runtime environment frames for call-by-value.

Phase	Frame contents
Before call	Global frame: $\{x : 5\}$
Call entry	Global frame: $\{x : 5\}$; new frame for f : $\{a : 5\}$
Inside body	Frame f : assignment $a = a + 1$ updates local frame to $\{a : 6\}$
After return	Function frame destroyed; global frame remains $\{x : 5\}$

Source program:

```

1 x = 5;
2
3 def f(a) {
4     a = a + 1;
5 }
```

```

6
7 f(x);

```

```

Before the call
Global scope:  x → 5

During the call
Global scope:  x → 5
Function scope (f):  a → 5, then a → 6

After the call
Global scope:  x → 5

```

Figure 5.1. Value parameter passing: the parameter *a* receives a copy of *x*, so mutating *a* inside *f* does not affect *x* in the caller.

A parameter receives a copy of the argument value rather than a shared reference, modifications to *a* inside the function do not affect *x* in the caller environment. This demonstrates standard call-by-value semantics and avoids aliasing effects associated with reference-based mechanisms.

5.5 Type Error Reporting

Type errors are reported with source positions so that the user can locate the problem quickly. The format used by the system is:

```
[line X, col Y] TypeError: message
```

Examples include assigning a *String* to an *Integer* variable, using a non-Boolean condition in an *if* statement, or supplying the wrong argument type to a function.

Table 5.5. Representative type errors detected by the static type checker.

Invalid program	Reason	Detection stage
<code>x = 5; x = "hello";</code>	Inconsistent variable type	Type checker
<code>3 + 4.0</code>	Mixed arithmetic types	Type checker
<code>if (3) { ... }</code>	Condition not Boolean	Type checker
<code>print()</code>	Wrong argument count	Type checker
<code>x = print(5);</code>	Void assigned to variable	Type checker
<code>true == false</code>	Boolean comparison unsupported	Type checker

5.6 Example Type Inference Traces

Consider the following program fragment:

```

1 x = 3;
2 y = 4;
3 z = x + y;
4 print(z);

```

The checker infers:

- `x`: Integer (from literal 3)
- `y`: Integer (from literal 4)
- `x + y`: Integer (Integer + Integer $\rightarrow$ Integer)
- `z`: Integer

For a float example using dot-suffixed operators:

```

1 def addf(a, b) {
2     return a +. b;
3 }
4
5 result = addf(2.0, 3.5);

```

The first function call infers `a`: Float and `b`: Float. The operator `+.` requires both operands to be Float, so the function return type is inferred as Float. The variable `result` is also inferred as Float.

5.7 Static Semantics Summary

Table ?? summarises the key typing rules enforced by the type checker.

Table 5.6. Static semantics summary: constructs and their typing rules.

Construct	Rule
Integer arithmetic (+, −, *, /)	$\text{Integer} \times \text{Integer} \rightarrow \text{Integer}$
Float arithmetic (+., −., *., /.)	$\text{Float} \times \text{Float} \rightarrow \text{Float}$
Equality / inequality (==, !=)	Same numeric type $\rightarrow$ Boolean; Boolean/String operands rejected
Assignment	RHS type must match the variable's inferred type
if / while condition	Condition expression must be Boolean
Function return	All return expressions must share one type; that type becomes the function type

CHAPTER 6

SYSTEM ARCHITECTURE AND IMPLEMENTATION

This chapter describes how the implementation is organised: the end-to-end processing flow, repository layout, technology choices, and the main components that realise static analysis and execution (type checker, interpreter, and user interfaces). Detailed treatment of lexical analysis, the symbol table, and parsing appears in Chapters 3 and 4; Chapter 5 presents the type-inference rules that the type checker implements.

6.1 Compiler pipeline

The implementation follows four processing phases: lexical analysis, parsing, static type checking, and interpretation. The shared pipeline exposes five labelled outputs because the parsing phase produces two inspectable artefacts: a grammar-oriented parse tree and an executable AST. Lexical and syntactic processing are kept separate from semantic analysis and execution, which simplifies testing and localises errors to the appropriate stage.

The overall flow of data through the system is shown in Figure ??.

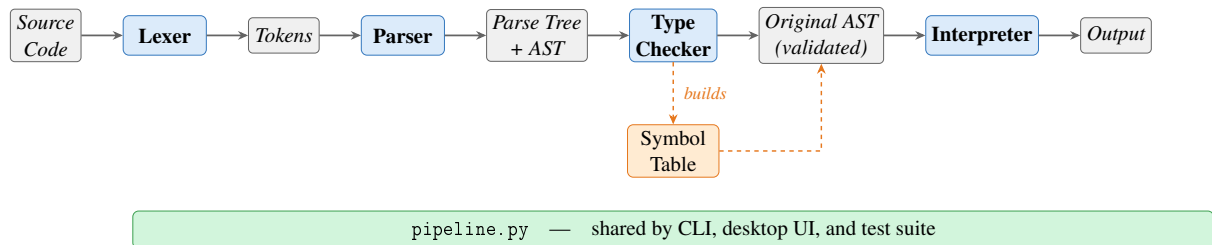


Figure 6.1. Front-end and interpreter pipeline overview.

Source characters are scanned by the lexer into a token stream. The parser consumes tokens and builds an AST that reflects program structure (expressions, statements, and control flow). The shared pipeline also renders a separate parse-tree view from the parsed program so that grammar structure can be inspected explicitly in the CLI, UI, and report. The type checker then walks the AST, populates and uses the symbol table, infers types, and validates scope and type rules. The pipeline does not construct a separate checked-AST object; instead, the original AST proceeds to interpretation only after successful type checking. The interpreter evaluates that validated tree using ordinary Python values; it does not re-run the type rules. The

prior static pass guarantees type safety properties (for example, no mixed numeric operators), but runtime faults that depend on concrete values (such as division by zero) can still occur during execution. All stages are orchestrated through `pipeline.py`, which is shared by the command-line runner, the desktop UI, and the test suite.

Table ?? shows a representative error for each stage.

Table 6.1. Representative errors detected at each pipeline stage

Stage	Example error	Effect
Lexer	Unrecognised character (#)	LexerError reported; pipeline halts
Parser	Missing closing)	ParseError reported; pipeline halts
Type checker	3 + 4.0 (mixed numeric types)	TypeError reported; interpreter not reached
Interpreter	Integer division by zero	RuntimeError reported at execution time

6.1.1 End-to-end pipeline walkthrough

The following example traces the complete execution of a five-statement program through the exported pipeline views and execution stages, illustrating the correspondence between source text, tokens, parse tree, abstract syntax tree, inferred types, and runtime output.

Source program:

```

1 a = 3;
2 b = 4;
3 print(a + b);
4 c = 1.5;
5 print(c +. c);

```

Stage 1 — Lexer output (token stream):

```

1 IDENTIFIER 'a' line=1 col=1
2 ASSIGN '=' line=1 col=3
3 INT_LITERAL '3' line=1 col=5
4 SEMICOLON ';' line=1 col=6
5 IDENTIFIER 'b' line=2 col=1
6 ASSIGN '=' line=2 col=3
7 INT_LITERAL '4' line=2 col=5
8 SEMICOLON ';' line=2 col=6
9 IDENTIFIER 'print' line=3 col=1
10 LPAREN '(' line=3 col=6
11 IDENTIFIER 'a' line=3 col=7
12 PLUS '+' line=3 col=9
13 IDENTIFIER 'b' line=3 col=11

```

```

14  RPAREN ')' line=3 col=12
15  SEMICOLON ';' line=3 col=13
16  IDENTIFIER 'c' line=4 col=1
17  ASSIGN '=' line=4 col=3
18  FLOAT_LITERAL '1.5' line=4 col=5
19  SEMICOLON ';' line=4 col=8
20  IDENTIFIER 'print' line=5 col=1
21  LPAREN '(' line=5 col=6
22  IDENTIFIER 'c' line=5 col=7
23  PLUS_DOT '+.' line=5 col=9
24  IDENTIFIER 'c' line=5 col=12
25  RPAREN ')' line=5 col=13
26  SEMICOLON ';' line=5 col=14
27  EOF '' line=5 col=15

```

Stage 2 — Parser output (ASCII branch parse tree):

```

1  program
2  |-- statement
3  | '-- assignment
4  | |-- IDENTIFIER(a)
5  | |-- '='
6  | |-- expr
7  | | '-- additive
8  | | '-- term
9  | | '-- factor
10 | | '-- INT_LITERAL(3)
11 | '-- ';'
12 |-- statement
13 | '-- assignment
14 | |-- IDENTIFIER(b)
15 | |-- '='
16 | |-- expr
17 | | '-- additive
18 | | '-- term
19 | | '-- factor
20 | | '-- INT_LITERAL(4)
21 | '-- ';'
22 |-- statement
23 | '-- expr_stmt
24 | |-- expr
25 | | '-- additive
26 | | '-- term
27 | | '-- factor
28 | | '-- function_call
29 | | |-- IDENTIFIER(print)
30 | | |-- '('
31 | | |-- args
32 | | | '-- expr
33 | | | '-- additive
34 | | | |-- additive
35 | | | | '-- term
36 | | | | '-- factor

```

```

37 | | | '-- IDENTIFIER(a)
38 | | | |-- '+'
39 | | | '-- term
40 | | | '-- factor
41 | | | '-- IDENTIFIER(b)
42 | | '-- ')
43 | '-- ';'
44 |-- statement
45 | '-- assignment
46 | |-- IDENTIFIER(c)
47 | |-- '='
48 | |-- expr
49 | | '-- additive
50 | | '-- term
51 | | '-- factor
52 | | '-- FLOAT_LITERAL(1.5)
53 | '-- ';'
54 |-- statement
55 | '-- expr_stmt
56 | |-- expr
57 | | '-- additive
58 | | '-- term
59 | | '-- factor
60 | | '-- function_call
61 | | |-- IDENTIFIER(print)
62 | | |-- '('
63 | | |-- args
64 | | | '-- expr
65 | | | '-- additive
66 | | | |-- additive
67 | | | | '-- term
68 | | | | '-- factor
69 | | | | '-- IDENTIFIER(c)
70 | | | |-- '+'
71 | | | '-- term
72 | | | '-- factor
73 | | | '-- IDENTIFIER(c)
74 | | '-- ')
75 | '-- ';'
76 '-- EOF

```

Stage 3 — Parser output (abstract syntax tree):

```

1 Program [5 statement(s)]
2   Assign 'a' =
3     Literal 3 (int)
4   Assign 'b' =
5     Literal 4 (int)
6   FunctionCall 'print' [1 arg(s)]
7     arg[0]:
8       BinaryOp '+'
9         left:
10          Identifier 'a'

```

```

11         right:
12             Identifier 'b'
13     Assign 'c' =
14         Literal 1.5 (float)
15     FunctionCall 'print' [1 arg(s)]
16     arg[0]:
17         BinaryOp '+.'
18         left:
19             Identifier 'c'
20         right:
21             Identifier 'c'

```

Stage 4 — Type checker output (inferred type environment):

Name	Inferred type
a	Integer
b	Integer
c	Float

The symbol table records only named symbols (a, b, and c). The type checker additionally infers that the sub-expression `a + b` has type `Integer` (`Integer + Integer → Integer`) and that `c +.` `c` has type `Float` (`Float +. Float → Float`) during AST traversal, but sub-expression types are not stored in the symbol table.

Stage 5 — Interpreter output:

```

1 7 : Integer
2 3.0 : Float

```

The example demonstrates the full data flow: source characters are tokenised by the lexer, structured by the parser into both an inspectable parse tree and an executable AST, validated by the type checker, and finally evaluated by the interpreter to produce typed output. It also shows that the ASCII branch parse tree preserves the concrete operator spellings `'+'` and `'+. '`, making the distinction between integer and float arithmetic visible in the exported parser view.

Float operator walkthrough

To illustrate how the dot-suffixed float operators flow through each pipeline stage, consider a second example:

Source program:

```

1 x = 2.0;
2 y = 3.5;
3 print(x +. y);

```


Stage 1 — Lexer output (token stream):

```
1 IDENTIFIER 'x' line=1 col=1
2 ASSIGN '=' line=1 col=3
3 FLOAT_LITERAL '2.0' line=1 col=5
4 SEMICOLON ';' line=1 col=8
5 IDENTIFIER 'y' line=2 col=1
6 ASSIGN '=' line=2 col=3
7 FLOAT_LITERAL '3.5' line=2 col=5
8 SEMICOLON ';' line=2 col=8
9 IDENTIFIER 'print' line=3 col=1
10 LPAREN '(' line=3 col=6
11 IDENTIFIER 'x' line=3 col=7
12 PLUS_DOT '+.' line=3 col=9
13 IDENTIFIER 'y' line=3 col=12
14 RPAREN ')' line=3 col=13
15 SEMICOLON ';' line=3 col=14
16 EOF '' line=3 col=15
```

Note that the lexer emits PLUS_DOT (not PLUS) for +. , distinguishing float addition from integer addition at the token level. Note also that print is tokenised as an ordinary IDENTIFIER — it is not a reserved keyword in this language.

Stage 3 — Parser output (abstract syntax tree):

```
1 Program [3 statement(s)]
2   Assign 'x' =
3     Literal 2.0 (float)
4   Assign 'y' =
5     Literal 3.5 (float)
6   FunctionCall 'print' [1 arg(s)]
7     arg[0]:
8       BinaryOp '+.'
9         left:
10           Identifier 'x'
11         right:
12           Identifier 'y'
```

The AST records the operator as +. rather than +, preserving the type distinction through to semantic analysis.

Stage 4 — Type checker output (inferred type environment):

Name	Inferred type
x	Float
y	Float

The type checker verifies that both operands of +. are Float. Had either operand been Integer, a TypeError would be reported before execution.

Stage 5 — Interpreter output:

```
1 5.5 : Float
```

This second walkthrough demonstrates that the dot-suffixed operators are visible at every pipeline stage — from the lexer token `PLUS_DOT` to the AST node `BinaryOp '+'` to the final typed output — ensuring that integer and float arithmetic are kept strictly separate throughout the system.

6.1.2 Entry points and shared pipeline

All stages are orchestrated by `components/pipeline.py`, which is shared by the command-line runner, the desktop UI, and the test suite. From the project root, the CLI and desktop UI can be launched as follows:

```
1 # macOS / Linux
2 python3 main.py # run CLI (built-in sample or script file)
3 python3 ui.py # open desktop workbench
4
5 # Windows
6 python main.py
7 python ui.py
```

The CLI prints five labelled stage outputs in order: tokens, parse tree, AST text, type table, and execution output. The desktop workbench uses the same pipeline internally, presenting each stage in a separate tab for inspection.

6.2 Project structure

The main source files and their responsibilities are summarised in Table ??.

Table 6.2. Main source files

File	Responsibility
<code>main.py</code>	Command-line runner for sample code or a script file
<code>ui.py</code>	Desktop UI for editing and running a script
<code>components/tokens.py</code>	Token classes and token categories
<code>components/lexica.py</code>	Lexical analysis
<code>components/ast_nodes.py</code>	AST node definitions
<code>components/parser.py</code>	Recursive-descent parser
<code>components/parse_tree_printer.py</code>	Parse-tree renderer for inspection
<code>components/ast_printer.py</code>	AST renderer for debugging and reporting
<code>components/symbol_table.py</code>	Scoped symbol table and semantic types
<code>components/type_checker.py</code>	Static type checker and inference
<code>components/interpreter.py</code>	Tree-walking interpreter
<code>components/pipeline.py</code>	Shared pipeline for CLI, UI, and tests

6.3 Technology stack

The project uses Python and standard-library tools only (Table ??), including `dataclasses` for AST structures, `tkinter` for the desktop interface, and `unittest` for automated verification (???)

Table 6.3. Technology stack

Tool	Purpose
Python 3.10+	Core implementation language
<code>dataclasses</code>	AST and runtime helper structures
<code>tkinter</code>	Desktop graphical interface
<code>unittest</code>	Automated testing

6.4 Type checker

The `TypeChecker` class implements the inference and checking rules described in Chapter 5. It traverses the AST before execution, uses the symbol table to record variable and function information, infers types on first assignment, validates arithmetic and comparison expressions, enforces Boolean conditions for control flow, fixes function parameter types for called functions from the first call site and validates all subsequent calls against those types (or infers from body usage for uncalled functions), and infers function return types from `return` statements. Programs that fail these rules are rejected before interpretation.

6.4.1 Scope of the implemented type system

In scope terms, this component turns a syntactically valid AST into a semantically valid program:

- static typing rules for arithmetic, comparison, assignment, control flow, and functions,
- type checking for expressions and statements,
- variable type inference from initial binding,
- function parameter type fixation for called functions at the first call site, with consistent validation of all subsequent calls (or body-based inference for uncalled functions),
- function return type inference from return statements,
- source-positioned type errors when a program violates a typing rule.

6.4.2 Static-vs-runtime function scope model

The static checker and runtime interpreter intentionally use different environment-parent links for function calls.

- During type checking, a function body is checked in a child scope of the global scope, because parameter and return constraints are validated against the globally registered function signatures.
- During execution, a function call frame is created with the caller environment as its parent, so ordinary dynamic variable lookup proceeds from local bindings to caller-visible bindings.

For this project's feature set, both models enforce the same intended language rules for typed assignments, control flow, and first-call parameter fixation. The distinction is documented here to avoid implying that both stages use identical scope-parent construction.

6.5 Interpreter

The interpreter is a tree-walking evaluator over the original AST after successful type checking. It maintains nested runtime environments for variables and function calls. Assignments write

through the current or nearest enclosing scope; `if` selects a branch; `while` repeats until its condition becomes false; function calls bind arguments by value and return through an internal return mechanism.

The built-in `print()` function outputs both value and runtime type. All four types are supported:

```
1 print(42);  
2 print(3.14);  
3 print(true);  
4 print("plc");
```

These produce the output lines `42 : Integer`, `3.14 : Float`, `true : Boolean`, and `"plc" : String` respectively.

6.5.1 Scope of the implemented runtime

The runtime stage provides observable behaviour and integrates with the shared pipeline:

- assignment execution,
- `if` execution,
- `while` execution,
- function execution with call-by-value parameter passing,
- `print()` execution with immediate output,
- execution behind the same `run_pipeline` path used by the CLI, UI, and tests.

6.6 Graphical user interface

The desktop workbench (`ui.py`) is implemented with Python's `tkinter` standard library and requires no additional dependencies. It uses a horizontal split layout with a source editor on the left and pipeline outputs on the right.

Toolbar. A top toolbar provides *Open Script*, *Run*, and *Clear Output* controls together with a status label.

Left panel (source editor). A monospaced `Text` widget with both horizontal and vertical scrollbars allows the user to enter or paste source code. The editor can also be populated from a file and is pre-loaded with the sample program on first launch.

Right panel (output tabs). A Notebook widget presents six scrollable read-only tabs:

- *Execution Output* — the `print()` values produced by stage 5, formatted as `value : Type`;
- *Tokens* — the token stream from stage 1, one token per line with type, lexeme, and source position;
- *Parse Tree* — a grammar-oriented ASCII branch rendering of stage 2;
- *AST* — the indented executable tree from stage 3;
- *Type Table* — the symbol table from stage 4 showing each name, kind, inferred type, initialisation state, and parameter list;
- *Errors* — populated only when a pipeline stage fails and cleared automatically after a successful run.

Error handling. If any pipeline stage raises an exception, the error message is written exclusively to the *Errors* tab and the remaining tabs are cleared. No modal dialogs are used, so the user can edit and rerun without dismissing a popup.

Keyboard shortcuts. F5 and Ctrl+Enter are both bound to the *Run* action, allowing rapid edit-run cycles without reaching for the mouse.

6.6.1 Screenshots

Figures ?? and ?? show the desktop workbench and CLI output.

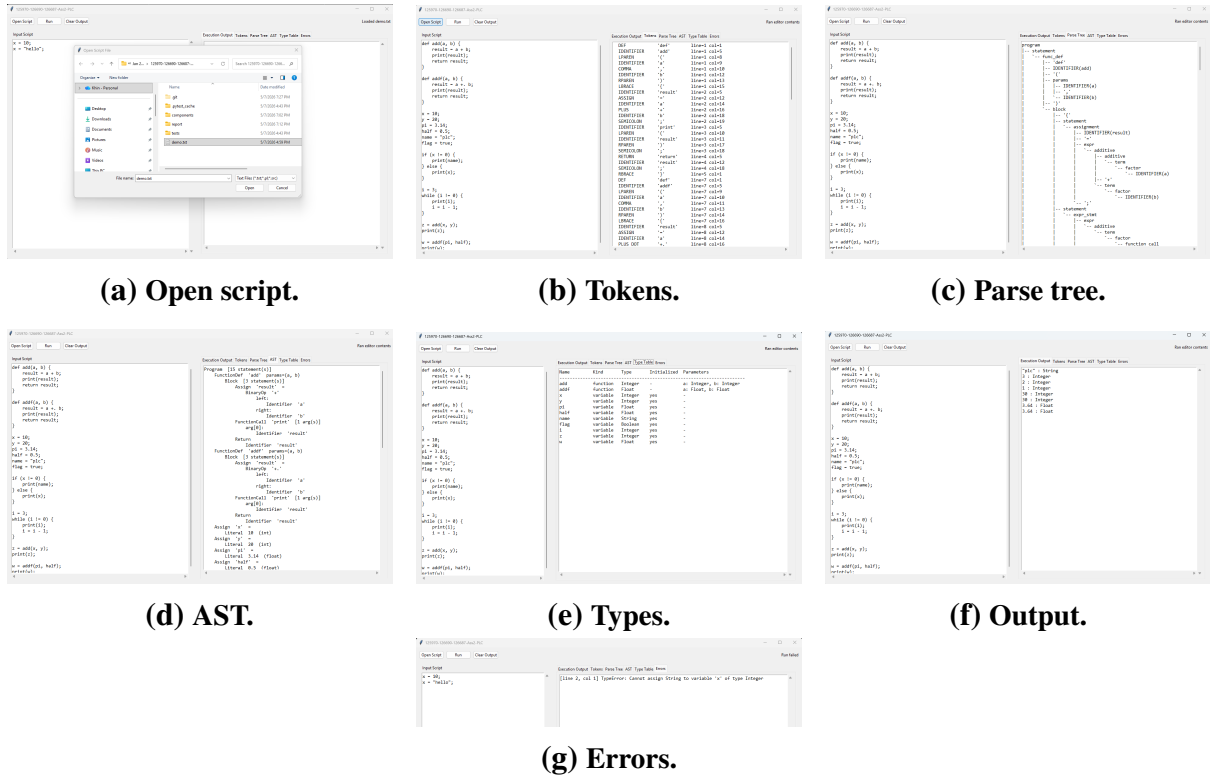


Figure 6.2. Desktop workbench tabs.

6.7 Command-line interface

The CLI runner (`main.py`) accepts an optional script path; without one it runs the built-in sample and prints five labelled stages to stdout. Errors are reported cleanly to stderr: if the specified file does not exist, the runner prints `ERROR: File not found: <path>` and exits with code 1. If any pipeline stage raises a language error (`LexerError`, `ParseError`, `TypeCheckError`, or `InterpreterError`), the message is printed as `ERROR: [line X, col Y] <ErrorType>: <detail>` and the process exits with code 1. In both cases no Python traceback is shown to the user.

(a) Tokens.

(b) Parse tree.

(c) AST.

(d) Types and output.

(e) CLI error.

Figure 6.3. CLI stages and error handling.

CHAPTER 7

TESTING AND VERIFICATION

7.1 Compiler-Theory Test Taxonomy

To align practical validation with compiler-theory reporting conventions, test evidence is grouped into lexical, syntax, semantic/type, and runtime classes, with both valid and invalid representatives.

Table 7.1. Formal test taxonomy used in this project

Category	Valid case	Invalid case	Expected stage
Lexical analysis	Tokenise identifiers, literals, keywords	Bare !, unterminated string	Lexer
Syntax analysis	Parse precedence-correct expressions and blocks	Missing semicolon, unmatched parenthesis	Parser
Static semantics	Well-typed assignment and function calls	Mixed numeric operators, non-Boolean condition	Type checker
Function typing	Consistent parameter and return types	Wrong argument type, inconsistent returns	Type checker
Runtime semantics	Correct arithmetic and loop execution	Division by zero	Interpreter

7.2 Manual Testing

Prior to the construction of the automated suite, each language feature was exercised manually by running the full pipeline and inspecting stage outputs. Table ?? summarises the categories covered.

Table 7.2. Representative manually tested categories

Category	Program Behaviour	Expected Result
Arithmetic	Evaluate integer and float expressions separately	Correct value and inferred type
If-then-else	Choose one branch from a Boolean condition	Only matching branch runs
While-loop	Print during repeated execution	One output line per iteration
Functions	Infer parameters and return type	Valid call and correct return value
Type mismatch	Reassign incompatible type	Type error before runtime

7.2.1 *Lexer and Symbol Table*

The lexer was verified manually by running the full pipeline and inspecting Stage 1 token output for programs that exercise every token category: integer and float literals, Boolean literals, string literals, the five reserved keywords, two-character comparison operators (`==`, `!=`), single-character operators, and delimiters. Identifier scanning with keyword fallback was confirmed by checking that `true` and `false` are emitted as `BOOL_LITERAL` tokens, the five reserved words (`if`, `else`, `while`, `def`, `return`) as their respective keyword token types, and ordinary names including `print` as `IDENTIFIER` tokens. For example, the source fragment `x = 10 ;` produces the token sequence:

```

1 IDENTIFIER 'x' line=1 col=1
2 ASSIGN '=' line=1 col=3
3 INT_LITERAL '10' line=1 col=5
4 SEMICOLON ';' line=1 col=7

```

Error paths were also checked: supplying a bare `!` character raises a `LexerError` with a source position and a message suggesting `!=`, and an unterminated string literal raises a `LexerError` citing the opening line.

The symbol table was verified through the type checker and the Stage 4 output. Direct symbol-table API tests confirmed that duplicate declarations in the same table raise `SymbolTableError`. In normal source-language execution, reassignment to an existing variable is permitted when the assigned type matches the inferred variable type, and rejected as `TypeError` only when the type is inconsistent. Scope isolation was verified with nested blocks: a variable defined inside an `if` or `while` body is not visible in the outer scope after the block ends. The `format_table()` output was checked against the expected column layout

(name, kind, type, initialisation status, parameters) for both variable and function symbols.

7.2.2 *Parser, Parse Tree, and AST*

The parser was verified manually by running the full pipeline and inspecting both the Stage 2 parse-tree output and the Stage 3 AST output. The following cases were checked:

- the parse-tree view contains grammar-oriented labels such as `program`, `statement`, `assignment`, and `expr`,
- arithmetic expressions with mixed precedence (e.g. `x + y * 2`) place multiplication in a deeper subtree than addition,
- `x = 5;` produces an `Assign` node, while `add(1,2);` produces a statement-level `FunctionCall`, confirming the IDENTIFIER+ASSIGN lookahead,
- `if` with and without an `else` clause yields an `If` node whose `else_block` is present only when required,
- function definitions store only parameter names in `FunctionDef`, with no type annotations at this stage,
- string literals in `Literal` nodes have surrounding double-quotes stripped, while runtime output restores them for `String` values,
- missing semicolons, unmatched parentheses, and unexpected tokens all produce source-positioned `ParseError` messages.

7.2.3 *Arithmetic Expressions*

Arithmetic expressions were tested with integer-only and float-only programs to verify operator precedence and strict type separation. The expression `x + y * 2` confirmed that multiplication binds more tightly than addition, and mixing integer and float operands correctly produced a type error, confirming the no-overloading design. The test suite explicitly verifies that mixed-type arithmetic is rejected:

- `1 + 2` — valid Integer arithmetic.
- `2.0 + . 3.5` — valid Float arithmetic.

- `5.0 -. 1.5, 2.0 *. 3.0, 9.0 /. 4.0` — valid Float arithmetic for the other dot-suffixed operators.
- `1 + 2.5` — `TypeError: operator + requires two Integer operands`.
- `2 +. 3` — `TypeError: operator +. requires two Float operands`.
- `10 / 0` and `10.0 /. 0.0` — `InterpreterError: division by zero, raised at runtime`.
- `1.0 +. 2.0 *. 3.0` — evaluates to `7.0` confirming `*. binds more tightly than +.`

These rejection tests are the primary evidence for the no-overloading requirement. In a promotion-based language, `1 + 2.5` and `2 +. 3` would silently succeed; here, both are rejected as `TypeError`s, showing that each operator token has a single fixed type signature and that no implicit coercion occurs. The suite includes 7 dedicated mixed-type and wrong-type operator tests (Table ??, `TypeCheckerTests`), so this guarantee is verified mechanically rather than asserted informally.

7.2.4 Boolean Expressions

Boolean expressions were tested inside `if` and `while` conditions. The type checker correctly enforced that both operands of `==` and `!=` must be arithmetic; supplying a `Boolean` or `String` operand produced a source-positioned `TypeCheckError` before any execution.

7.2.5 Assignment Statements

Assignments were tested for first-use type inference and reassignment consistency. For example, assigning 5 and then "hello" to the same variable correctly produced a type error, confirming that the inferred type is fixed after the initial binding.

7.2.6 If-Then-Else

Conditional programs were constructed to verify that exactly one branch executes for a given condition and that neither branch is entered when the condition evaluates to the opposite truth value. Non-Boolean conditions were also supplied and confirmed to be rejected by the type checker before execution.

7.2.7 While-Loop

While-loops were tested with an integer countdown from three to zero. The results confirmed that the loop body executes once per iteration and terminates when the condition becomes false,

with `print()` producing one output line per pass rather than accumulating output until loop exit.

7.2.8 *Functions*

Function tests covered value parameter passing (call-by-value), local scope execution, and return value propagation. For called functions, the type checker correctly fixed parameter types from the first call site and enforced that signature for all subsequent calls to the same function.

7.2.9 *Print*

The built-in `print()` function was tested with all supported runtime types. The output includes the printed value together with its type, such as `3.14 : Float` and `"plc" : String`.

7.2.10 *Unary Minus*

The unary minus operator was verified for integer literals (`x = -5`), float literals (`y = -. 3.14`), negated float variables (`y = -. x`), negated integer variables (`y = -x`), and negation inside a larger expression (`x = 3 + -2`). In each case the type checker assigned the correct type and the interpreter produced the expected value. Note that float negation uses the `-.` token, consistent with the Caml Light convention; using plain `-` on a `Float` operand is rejected by the type checker.

7.3 **Type Error Detection**

The checker was verified with invalid programs involving incompatible assignment, invalid arithmetic operands, wrong function arguments, and non-Boolean conditions. In each case the system stopped before execution and reported a source-positioned type error. Representative error messages are shown below.

```
1 # Assignment type mismatch
2 [line 2, col 1] TypeError: Cannot assign String to variable 'x' of type Integer
3
4 # Non-Boolean if condition
5 [line 1, col 4] TypeError: If condition must have type Boolean
6
7 # Wrong argument type for function
8 [line 5, col 9] TypeError: Argument 1 of 'add' must be Integer, got String
```

Table 7.3. Representative negative semantic test cases

Program fragment	Expected result
<code>1 + 2.0</code>	<code>TypeError</code>
<code>x = 5; x = true;</code>	<code>TypeError</code>
<code>if (3) { }</code>	<code>TypeError</code>
<code>print(1, 2)</code>	<code>TypeError</code>
<code>"a" + "b"</code>	<code>TypeError</code>
<code>true == false</code>	<code>TypeError</code>

These negative cases provide direct evidence that the type system enforces the no-overloading and same-type-comparison rules, rather than relying on positive cases alone.

7.4 Automated Test Suite

The automated regression suite is located in `tests/test_pipeline.py` and uses Python's built-in `unittest` framework. The tests are organised into five classes, each targeting a distinct compiler stage.

Table 7.4. Automated test coverage by class

Test class	Stage covered	Tests
<code>LexerTests</code>	Tokenisation (<code>lexica.py</code>)	12
<code>SymbolTableTests</code>	Scoped symbol table (<code>symbol_table.py</code>)	7
<code>ParserTests</code>	Parsing, precedence, parse-tree/AST output	7
<code>TypeCheckerTests</code>	Static type inference (<code>type_checker.py</code>)	22
<code>IntegrationTests</code>	Full pipeline execution	29
Total		77

LexerTests verify that each token category (integer, float, Boolean, string, keywords, identifiers, two-character operators, and the four float-arithmetic operators `+`, `-`, `*`, `/`) is produced correctly, that float literals such as `3.14` are not confused with integer followed by a dot operator, that source line numbers are tracked across newlines, and that illegal characters and unterminated strings raise `LexerError`.

ParserTests verify structural correctness of parsing. The 7 tests cover missing-semicolon and unclosed-brace error reporting, integer and float operator precedence, left-associativity, parenthesised expressions overriding precedence, and availability of both parse-tree and AST renderings in the pipeline result.

SymbolTableTests exercise the symbol table directly, without going through the full pipeline.

They cover variable definition and lookup, duplicate definitions for variables and functions, parent-scope lookup through `child_scope()`, undefined-symbol errors, and `format_table()` output for variable and function rows.

TypeCheckerTests verify static type inference and error detection. The 22 tests cover integer and float arithmetic, integer floor division, strict type separation, string and Boolean type inference, assignment type mismatch, non-Boolean conditions, undefined variables, wrong argument count and type, and uncalled-function body checking for both invalid and valid cases.

IntegrationTests run programs through multiple pipeline stages and verify final output. Cases include unary negation on all numeric types, division by zero for both `/` and `/.`, both branches of `if/else`, while-loop countdown, `print()` with every type, value-parameter isolation, nested function calls, comparison results assigned and printed directly, variable reassignment, scope isolation, rejection of passing a void-returning function to `print()`, and rejection of a function with a non-total return.

The tests can be run with:

```
1 # macOS / Linux
2 python3 -m unittest discover -s tests -v
3
4 # Windows
5 python -m unittest discover -s tests -v
```

All 77 tests pass.

7.5 Error Handling

The language system reports errors at four stages:

- lexical errors for malformed characters or unterminated strings,
- parse errors for invalid syntax such as missing semicolons,
- type errors for programs that violate static typing rules,
- runtime errors for division by zero and other execution failures.

All error messages include a source position in the form `[line X, col Y]` to aid debugging.

7.6 Limitations and Future Work

The implemented language intentionally restricts several advanced programming-language features to focus on core compiler construction principles and static type inference.

The current implementation does not support recursive functions, arrays, records, polymorphic typing, closures, or garbage collection. Type inference is syntax-directed and deterministic rather than based on full unification algorithms such as Hindley–Milner inference. The execution model is interpreted directly from the AST rather than compiled into bytecode or machine code.

Future extensions may include recursive function support, richer data structures, additional control-flow constructs, optimisation passes, and compilation to a lower-level intermediate representation.

CHAPTER 8

CONCLUSION

This report has presented a complete statically-typed interpreted language with four primitive types — Integer, Float, Boolean, and String — together with assignment, control flow, user-defined functions, and a built-in `print()` function. The implementation integrates lexical analysis, parsing, static semantic analysis, and interpretation into a coherent four-stage pipeline. Types are inferred without explicit declarations, and ill-typed programs are rejected before execution.

A key design decision is the use of distinct operator tokens for integer and floating-point arithmetic, following the Caml Light convention. This eliminates implicit numeric promotion and ensures that each operator has exactly one type signature, yielding deterministic type inference. The system also exposes both a grammar-oriented parse-tree view and an executable AST, while the automated test suite of 77 tests confirms correct behaviour across lexical analysis, symbol-table handling, parsing, type checking, and end-to-end execution.

The result is a clear and modular language-processing pipeline with strong static guarantees and inspectable intermediate forms. Future work could extend the language with recursive functions, richer data structures, and code generation.

APPENDIX A: SOURCE CODE

This appendix includes representative source files that highlight the core implementation of the language processor. The selected listings focus on the principal implementation stages: lexical analysis, parsing, static semantic analysis, interpretation, and program entry. The complete codebase is available in the linked repository.

Source code repository:

<https://github.com/The-Token-Trio/125970-126690-126687-Assignment2-PLC>

Video walkthrough:

<https://youtu.be/sR2JRM6eYzM>

APPENDIX B: TEAM MEMBERS AND CONTRIBUTIONS

Name	Student ID	Contributions
Aye Khin Khin Hpone (Yolanda Lim)	st125970	Static typing and type checking; tree-walking interpreter (assignment, if, while, function, <code>print()</code>); unary minus inference/execution; Desktop UI; CLI; pipeline integration; automated testing (77 tests)
Applegate T. Tun Oo	st126690	Lexer implementation (scanning, tokenisation, line/column tracking, error reporting); token definitions; keywords, operators, identifiers, and literals; symbol table (variable/function/type storage)
Win Htut Naing	st126687	Arithmetic expressions (incl. unary minus); boolean expressions; assignment statements; if-then-else; while-loop; function definitions and calls; <code>print()</code> syntax; parse-tree and AST construction

A.1 Representative Source Files

Listing 8.1: main.py

```
from __future__ import annotations

import argparse
import sys
from pathlib import Path

from components.interpreter import InterpreterError
from components.lexica import LexerError
from components.parser import ParseError
from components.pipeline import format_stage_output, run_pipeline
from components.type_checker import TypeCheckError

DEFAULT_SOURCE = """
def add(a, b) {
    result = a + b;
    print(result);
    return result;
}

def addf(a, b) {
    result = a +. b;
    print(result);
    return result;
}

x = 10;
y = 20;
pi = 3.14;
half = 0.5;
name = "plc";
flag = true;

if (x != 0) {
    print(name);
} else {
    print(x);
}

i = 3;
while (i != 0) {
    print(i);
    i = i - 1;
}

z = add(x, y);
print(z);

w = addf(pi, half);
print(w);
"""

def run_source(source_code: str) -> None:
    try:
        result = run_pipeline(source_code)
        print(format_stage_output(result))
    except (LexerError, ParseError, TypeCheckError, InterpreterError) as error:
        print(f"ERROR: {error}", file=sys.stderr)
        sys.exit(1)

def _parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Run the programming language pipeline.")
    parser.add_argument("script", nargs="?", help="Optional path to a source file to run")
```

```

        return parser.parse_args()

def main() -> None:
    args = _parse_args()
    if args.script:
        path = Path(args.script)
        if not path.exists():
            print(f"ERROR: File not found: {args.script}", file=sys.stderr)
            sys.exit(1)
        source_code = path.read_text(encoding="utf-8")
    else:
        source_code = DEFAULT_SOURCE
    run_source(source_code)

if __name__ == "__main__":
    main()

```

Listing 8.2: components/lexica.py

```

from __future__ import annotations

from typing import Iterator

from components.tokens import Token, TokenType

KEYWORDS: dict[str, TokenType] = {
    "if": TokenType.IF,
    "else": TokenType.ELSE,
    "while": TokenType.WHILE,
    "def": TokenType.DEF,
    "return": TokenType.RETURN,
    "true": TokenType.BOOL_LITERAL,
    "false": TokenType.BOOL_LITERAL,
}

# Arithmetic operators that may have a trailing '.' for the float variant
_ARITH_DOT: dict[str, tuple[TokenType, TokenType]] = {
    "+": (TokenType.PLUS, TokenType.PLUS_DOT),
    "-": (TokenType.MINUS, TokenType.MINUS_DOT),
    "*": (TokenType.STAR, TokenType.STAR_DOT),
    "/": (TokenType.SLASH, TokenType.SLASH_DOT),
}

SINGLE_CHAR_TOKENS: dict[str, TokenType] = {
    "=": TokenType.ASSIGN,
    "(": TokenType.LPAREN,
    ")": TokenType.RPAREN,
    "{": TokenType.LBRACE,
    "}": TokenType.RBRACE,
    ",": TokenType.COMMA,
    ";": TokenType.SEMICOLON,
}

class LexerError(Exception):
    pass

class Lexer:
    def __init__(self, source: str) -> None:
        self.source = source
        self.start = 0
        self.current = 0
        self.line = 1
        self.column = 1

```

```

self.token_column = 1

def tokenize(self) -> list[Token]:
    tokens: list[Token] = []
    while not self._is_at_end():
        self.start = self.current
        self.token_column = self.column
        token = self._scan_token()
        if token is not None:
            tokens.append(token)
        tokens.append(Token(TokenType.EOF, "", self.line, self.column))
    return tokens

def _scan_token(self) -> Token | None:
    char = self._advance()

    # Whitespace handling
    if char in {" ", "\t", "\r"}:
        return None
    if char == "\n":
        self.line += 1
        self.column = 1
        return None

    # Two-char operators
    if char == "=":
        if self._match("=="):
            return self._make_token(TokenType.EQUAL_EQUAL)
        return self._make_token(TokenType.ASSIGN)
    if char == "!":
        if self._match("!="):
            return self._make_token(TokenType.BANG_EQUAL)
        raise LexerError(self._error_message("Unexpected '!'. Did you mean '!='?"))

    # Arithmetic operators: + - * / (with optional trailing '.' for float variant)
    arith = _ARITH_DOT.get(char)
    if arith is not None:
        int_tok, float_tok = arith
        # Unary-minus lookahead: '-' followed by a digit is NOT '-' even if
        # next char is '.' -- we still need to check char after '.' is not digit
        # (to avoid consuming the '.' of a float literal like 3.14).
        # Rule: X. is a float operator only when the char after '.' is NOT a digit.
        if self._peek() == "." and not self._peek_next().isdigit():
            self._advance() # consume '.'
            return self._make_token(float_tok)
        return self._make_token(int_tok)

    # Single-char operators/delimiters
    single_char_token = SINGLE_CHAR_TOKENS.get(char)
    if single_char_token is not None:
        return self._make_token(single_char_token)

    if char == '"':
        return self._string()

    if char.isdigit():
        return self._number()

    if self._is_identifier_start(char):
        return self._identifier()

    raise LexerError(self._error_message(f"Unexpected character: {char!r}"))

def _string(self) -> Token:
    while self._peek() != '"' and not self._is_at_end():
        if self._peek() == "\n":
            self.line += 1
            self.column = 1
        self._advance()

```

```

        if self._is_at_end():
            raise LexerError(self._error_message("Unterminated string literal"))

        # Closing quote
        self._advance()
        return self._make_token(TokenType.STRING_LITERAL)

def _number(self) -> Token:
    while self._peek().isdigit():
        self._advance()

    is_float = False
    if self._peek() == "." and self._peek_next().isdigit():
        is_float = True
        self._advance()
        while self._peek().isdigit():
            self._advance()

    if is_float:
        return self._make_token(TokenType.FLOAT_LITERAL)
    return self._make_token(TokenType.INT_LITERAL)

def _identifier(self) -> Token:
    while self._is_identifier_part(self._peek()):
        self._advance()

    lexeme = self._current_lexeme()
    token_type = KEYWORDS.get(lexeme, TokenType.IDENTIFIER)
    return self._make_token(token_type)

def _current_lexeme(self) -> str:
    return self.source[self.start : self.current]

def _make_token(self, token_type: TokenType) -> Token:
    return Token(token_type, self._current_lexeme(), self.line, self.token_column)

def _advance(self) -> str:
    char = self.source[self.current]
    self.current += 1
    self.column += 1
    return char

def _match(self, expected: str) -> bool:
    if self._is_at_end():
        return False
    if self.source[self.current] != expected:
        return False

    self.current += 1
    self.column += 1
    return True

def _peek(self) -> str:
    if self._is_at_end():
        return "\0"
    return self.source[self.current]

def _peek_next(self) -> str:
    if self.current + 1 >= len(self.source):
        return "\0"
    return self.source[self.current + 1]

def _is_at_end(self) -> bool:
    return self.current >= len(self.source)

@staticmethod
def _is_identifier_start(char: str) -> bool:
    return char.isalpha() or char == "_"

@staticmethod

```

```

def _is_identifier_part(char: str) -> bool:
    return char.isalnum() or char == "_"

def _error_message(self, message: str) -> str:
    return f"[line {self.line}, col {self.token_column}] LexerError: {message}"

def iter_tokens(source: str) -> Iterator[Token]:
    yield from Lexer(source).tokenize()

```

Listing 8.3: components/parser.py

```

from __future__ import annotations

from typing import Optional

from components.tokens import Token, TokenType
from components.ast_nodes import (
    ASTNode,
    Program,
    Block,
    Assign,
    If,
    While,
    FunctionDef,
    FunctionCall,
    Return,
    BinaryOp,
    Literal,
    Identifier,
)

# -----
# Parser error
# -----

class ParseError(Exception):
    pass

# -----
# Parser
# -----

class Parser:
    """Recursive-descent parser.

    Usage:
        tokens = Lexer(source).tokenize()
        tree = Parser(tokens).parse() # returns Program node
    """

    def __init__(self, tokens: list[Token]) -> None:
        self._tokens = tokens
        self._pos = 0

    # -----
    # Public entry point
    # -----

    def parse(self) -> Program:
        """Parse the full token stream and return the root Program node."""
        line, col = self._peek().line, self._peek().column
        statements: list[ASTNode] = []
        while not self._is_at_end():
            statements.append(self._statement())
        return Program(statements=statements, line=line, column=col)

```

```

# -----
# Statements
# -----

def _statement(self) -> ASTNode:
    """Dispatch to the correct statement parser based on the current token."""
    tok = self._peek()

    if tok.token_type == TokenType.DEF:
        return self._func_def()

    if tok.token_type == TokenType.IF:
        return self._if_stmt()

    if tok.token_type == TokenType.WHILE:
        return self._while_stmt()

    if tok.token_type == TokenType.RETURN:
        return self._return_stmt()

    # assignment: IDENTIFIER "=" expr ";"
    if (
        tok.token_type == TokenType.IDENTIFIER
        and self._peek_next().token_type == TokenType.ASSIGN
    ):
        return self._assignment()

    # fallback: bare expression statement expr ";"
    node = self._expr()
    self._expect(TokenType.SEMICOLON, "Expected ';' after expression")
    return node

def _assignment(self) -> Assign:
    """IDENTIFIER '=' expr ';'"""
    name_tok = self._advance() # IDENTIFIER
    self._advance() # '='
    value = self._expr()
    self._expect(TokenType.SEMICOLON, "Expected ';' after assignment")
    return Assign(name=name_tok.lexeme, value=value,
                  line=name_tok.line, column=name_tok.column)

def _if_stmt(self) -> If:
    """'if' '(' expr ')' block ( 'else' block )?"""
    tok = self._advance() # 'if'
    self._expect(TokenType.LPAREN, "Expected '(' after 'if'")
    condition = self._expr()
    self._expect(TokenType.RPAREN, "Expected ')' after if condition")
    then_block = self._block()
    else_block: Optional[Block] = None
    if self._check(TokenType.ELSE):
        self._advance() # 'else'
        else_block = self._block()
    return If(condition=condition, then_block=then_block, else_block=else_block,
              line=tok.line, column=tok.column)

def _while_stmt(self) -> While:
    """'while' '(' expr ')' block"""
    tok = self._advance() # 'while'
    self._expect(TokenType.LPAREN, "Expected '(' after 'while'")
    condition = self._expr()
    self._expect(TokenType.RPAREN, "Expected ')' after while condition")
    body = self._block()
    return While(condition=condition, body=body,
                 line=tok.line, column=tok.column)

def _func_def(self) -> FunctionDef:
    """'def' IDENTIFIER '(' params? ')' block"""
    tok = self._advance() # 'def'
    name_tok = self._expect(TokenType.IDENTIFIER, "Expected function name after 'def'")
    self._expect(TokenType.LPAREN, "Expected '(' after function name")

```



```

        params = self._params()
        self._expect(TokenType.RPAREN, "Expected ')' after parameters")
        body = self._block()
        return FunctionDef(name=name_tok.lexeme, params=params, body=body,
                           line=tok.line, column=tok.column)

def _return_stmt(self) -> Return:
    """'return' expr ';'"""
    tok = self._advance() # 'return'
    expr = self._expr()
    self._expect(TokenType.SEMICOLON, "Expected ';' after return value")
    return Return(expr=expr, line=tok.line, column=tok.column)

def _block(self) -> Block:
    """{' statement* }'"""
    tok = self._expect(TokenType.LBRACE, "Expected '{'")
    statements: list[ASTNode] = []
    while not self._check(TokenType.RBRACE) and not self._is_at_end():
        statements.append(self._statement())
    self._expect(TokenType.RBRACE, "Expected '}' to close block")
    return Block(statements=statements, line=tok.line, column=tok.column)

# -----
# Parameters (function definition)
# -----

def _params(self) -> list[str]:
    """IDENTIFIER (',' IDENTIFIER)* -- returns list of param names."""
    params: list[str] = []
    if self._check(TokenType.IDENTIFIER):
        params.append(self._advance().lexeme)
        while self._check(TokenType.COMMA):
            self._advance() # ','
            tok = self._expect(TokenType.IDENTIFIER, "Expected parameter name after ',")
            params.append(tok.lexeme)
    return params

# -----
# Expressions (comparison is lowest precedence, then additive, then term)
# -----

def _expr(self) -> ASTNode:
    """additive (('==' | '!=') additive)? -- non-associative comparison"""
    node = self._additive()
    if self._check(TokenType.EQUAL_EQUAL) or self._check(TokenType.BANG_EQUAL):
        op_tok = self._advance()
        right = self._additive()
        node = BinaryOp(op=op_tok.lexeme, left=node, right=right,
                        line=op_tok.line, column=op_tok.column)
    return node

def _additive(self) -> ASTNode:
    """term (('+' | '-' | '+' | '-') term)*"""
    node = self._term()
    while self._check(TokenType.PLUS) or self._check(TokenType.MINUS) \
        or self._check(TokenType.PLUS_DOT) or self._check(TokenType.MINUS_DOT):
        op_tok = self._advance()
        right = self._term()
        node = BinaryOp(op=op_tok.lexeme, left=node, right=right,
                        line=op_tok.line, column=op_tok.column)
    return node

def _term(self) -> ASTNode:
    """factor (('*' | '/' | '*' | '/') factor)*"""
    node = self._factor()
    while self._check(TokenType.STAR) or self._check(TokenType.SLASH) \
        or self._check(TokenType.STAR_DOT) or self._check(TokenType.SLASH_DOT):
        op_tok = self._advance()
        right = self._factor()
        node = BinaryOp(op=op_tok.lexeme, left=node, right=right,

```

```

        line=op_tok.line, column=op_tok.column)

    return node

def _factor(self) -> ASTNode:
    """
    '-' factor (unary negation, desugared to 0 - factor)
    | INT_LITERAL | FLOAT_LITERAL | STRING_LITERAL | BOOL_LITERAL
    | IDENTIFIER ( '(' args? ')' )?
    | '(' expr ')'
    """
    tok = self._peek()

    # Unary minus '-' factor (integer negation, desugared to 0 - factor)
    if tok.token_type == TokenType.MINUS:
        op_tok = self._advance()
        operand = self._factor()
        zero = Literal(value=0, line=op_tok.line, column=op_tok.column)
        return BinaryOp(op="-", left=zero, right=operand,
                        line=op_tok.line, column=op_tok.column)

    # Unary minus-dot '-.' factor (float negation, desugared to 0.0 -. factor)
    if tok.token_type == TokenType.MINUS_DOT:
        op_tok = self._advance()
        operand = self._factor()
        zero_f = Literal(value=0.0, line=op_tok.line, column=op_tok.column)
        return BinaryOp(op="-. ", left=zero_f, right=operand,
                        line=op_tok.line, column=op_tok.column)

    # Integer literal
    if tok.token_type == TokenType.INT_LITERAL:
        self._advance()
        return Literal(value=int(tok.lexeme), line=tok.line, column=tok.column)

    # Float literal
    if tok.token_type == TokenType.FLOAT_LITERAL:
        self._advance()
        return Literal(value=float(tok.lexeme), line=tok.line, column=tok.column)

    # Boolean literal
    if tok.token_type == TokenType.BOOL_LITERAL:
        self._advance()
        return Literal(value=(tok.lexeme == "true"), line=tok.line, column=tok.column)

    # String literal (keep the surrounding quotes stripped)
    if tok.token_type == TokenType.STRING_LITERAL:
        self._advance()
        return Literal(value=tok.lexeme[1:-1], line=tok.line, column=tok.column)

    # Identifier -- could be a plain variable or a function call
    if tok.token_type == TokenType.IDENTIFIER:
        self._advance()
        if self._check(TokenType.LPAREN):
            # function call
            self._advance() # '('
            args = self._args()
            self._expect(TokenType.RPAREN, "Expected ')' after function arguments")
            return FunctionCall(name=tok.lexeme, args=args,
                               line=tok.line, column=tok.column)
        return Identifier(name=tok.lexeme, line=tok.line, column=tok.column)

    # Grouped expression '(' expr ')
    if tok.token_type == TokenType.LPAREN:
        self._advance() # '('
        node = self._expr()
        self._expect(TokenType.RPAREN, "Expected ')' to close grouped expression")
        return node

    raise ParseError(self._error_at(tok, f"Unexpected token: {tok.lexeme!r}"))

# -----

```

```

# Arguments (function call)
# -----

def _args(self) -> list[ASTNode]:
    """expr (',' expr)*"""
    args: list[ASTNode] = []
    if not self._check(TokenType.RPAREN):
        args.append(self._expr())
        while self._check(TokenType.COMMA):
            self._advance() # ','
            args.append(self._expr())
    return args

# -----
# Token navigation helpers
# -----

def _peek(self) -> Token:
    return self._tokens[self._pos]

def _peek_next(self) -> Token:
    if self._pos + 1 < len(self._tokens):
        return self._tokens[self._pos + 1]
    return self._tokens[-1] # EOF

def _advance(self) -> Token:
    tok = self._tokens[self._pos]
    if not self._is_at_end():
        self._pos += 1
    return tok

def _check(self, token_type: TokenType) -> bool:
    return self._peek().token_type == token_type

def _is_at_end(self) -> bool:
    return self._peek().token_type == TokenType.EOF

def _expect(self, token_type: TokenType, message: str) -> Token:
    if self._check(token_type):
        return self._advance()
    raise ParseError(self._error_at(self._peek(), message))

def _error_at(self, tok: Token, message: str) -> str:
    return f"[line {tok.line}, col {tok.column}] ParseError: {message}"

```

Listing 8.4: components/type_checker.py

```

from __future__ import annotations

from dataclasses import dataclass

from components.ast_nodes import (
    ASTNode,
    Assign,
    BinaryOp,
    Block,
    FunctionCall,
    FunctionDef,
    Identifier,
    If,
    Literal,
    Program,
    Return,
    While,
)

from components.symbol_table import FunctionSymbol, LanguageType, SymbolTable, SymbolTableError,
    VariableSymbol

```

```

class TypeCheckError(Exception):
    pass

@dataclass
class _FunctionState:
    node: FunctionDef
    body_checked: bool = False

@dataclass
class _FunctionContext:
    name: str
    return_type: LanguageType | None = None

class TypeChecker:
    _BUILTIN_PRINT = "print"

    def __init__(self) -> None:
        self.global_scope = SymbolTable()
        self._functions: dict[str, _FunctionState] = {}
        self._active_calls: list[str] = []

    def check(self, program: Program) -> SymbolTable:
        for statement in program.statements:
            if isinstance(statement, FunctionDef):
                self._register_function(statement)

        for statement in program.statements:
            if isinstance(statement, FunctionDef):
                continue
            self._check_statement(statement, self.global_scope, None)

        self._check_uncalled_functions()
        return self.global_scope

    def _register_function(self, node: FunctionDef) -> None:
        if node.name in self._functions:
            raise TypeCheckError(self._error_at(node, f"Function '{node.name}' is already defined"))

        placeholder_parameters = [(param_name, LanguageType.VOID) for param_name in node.params]
        try:
            self.global_scope.define_function(node.name, LanguageType.VOID, placeholder_parameters)
        except SymbolTableError as error:
            raise TypeCheckError(self._error_at(node, str(error))) from error
        self._functions[node.name] = _FunctionState(node=node)

    def _check_statement(
        self,
        node: ASTNode,
        scope: SymbolTable,
        function_context: _FunctionContext | None,
    ) -> None:
        if isinstance(node, Assign):
            value_type = self._infer_expr_type(node.value, scope)
            self._assign_variable_type(scope, node, value_type)
            return

        if isinstance(node, Return):
            if function_context is None:
                raise TypeCheckError(self._error_at(node, "'return' is only valid inside a function"))
            expr_type = self._infer_expr_type(node.expr, scope)
            if function_context.return_type is None:
                function_context.return_type = expr_type
            elif function_context.return_type != expr_type:
                raise TypeCheckError(
                    self._error_at(
                        node,
                        f"Function '{function_context.name}' returns both {function_context.return_type.value

```

```

        } and {expr_type.value}",
    )
    )
    return

if isinstance(node, If):
    condition_type = self._infer_expr_type(node.condition, scope)
    if condition_type != LanguageType.BOOLEAN:
        raise TypeCheckError(self._error_at(node.condition, "If condition must have type Boolean"))
    self._check_block(node.then_block, scope.child_scope(), function_context)
    if node.else_block is not None:
        self._check_block(node.else_block, scope.child_scope(), function_context)
    return

if isinstance(node, While):
    condition_type = self._infer_expr_type(node.condition, scope)
    if condition_type != LanguageType.BOOLEAN:
        raise TypeCheckError(self._error_at(node.condition, "While condition must have type Boolean"))
    self._check_block(node.body, scope.child_scope(), function_context)
    return

if isinstance(node, Block):
    self._check_block(node, scope.child_scope(), function_context)
    return

if isinstance(node, FunctionCall):
    self._infer_function_call_type(node, scope)
    return

raise TypeCheckError(self._error_at(node, f"Unsupported statement node: {type(node).__name__}"))

def _check_block(
    self,
    block: Block,
    scope: SymbolTable,
    function_context: _FunctionContext | None,
) -> None:
    for statement in block.statements:
        self._check_statement(statement, scope, function_context)

def _infer_expr_type(self, node: ASTNode, scope: SymbolTable) -> LanguageType:
    if isinstance(node, Literal):
        return self._literal_type(node)

    if isinstance(node, Identifier):
        try:
            symbol = scope.lookup(node.name)
        except SymbolTableError as error:
            raise TypeCheckError(self._error_at(node, str(error))) from error
        if not isinstance(symbol, VariableSymbol):
            raise TypeCheckError(self._error_at(node, f"'{node.name}' is not a variable"))
        return symbol.symbol_type

    if isinstance(node, FunctionCall):
        return self._infer_function_call_type(node, scope)

    if isinstance(node, BinaryOp):
        left_type = self._infer_expr_type(node.left, scope)
        right_type = self._infer_expr_type(node.right, scope)
        return self._infer_binary_type(node, left_type, right_type)

    raise TypeCheckError(self._error_at(node, f"Unsupported expression node: {type(node).__name__}"))

def _infer_function_call_type(self, node: FunctionCall, scope: SymbolTable) -> LanguageType:
    if node.name == self._BUILTIN_PRINT:
        return self._infer_builtin_print_type(node, scope)

    function_state = self._functions.get(node.name)
    if function_state is None:

```

```

        raise TypeCheckError(self._error_at(node, f"Undefined function: {node.name}"))
    if node.name in self._active_calls:
        raise TypeCheckError(self._error_at(node, f"Recursive function calls are not supported: {node.name}"))

    argument_types = [self._infer_expr_type(argument, scope) for argument in node.args]
    function_symbol = self._lookup_function_symbol(node)

    if len(argument_types) != len(function_symbol.parameters):
        raise TypeCheckError(
            self._error_at(
                node,
                f"Function '{node.name}' expects {len(function_symbol.parameters)} argument(s), got {len(argument_types)}",
            )
        )

    declared_parameter_types = [param_type for _, param_type in function_symbol.parameters]
    if all(param_type == LanguageType.VOID for param_type in declared_parameter_types):
        function_symbol.parameters = list(zip(function_state.node.params, argument_types, strict=False))
    else:
        for index, ((_, expected_type), actual_type) in enumerate(zip(function_symbol.parameters, argument_types, strict=False)):
            if expected_type != actual_type:
                raise TypeCheckError(
                    self._error_at(
                        node.args[index],
                        f"Argument {index + 1} of '{node.name}' must be {expected_type.value}, got {actual_type.value}",
                    )
                )

    if not function_state.body_checked:
        self._check_function_body(function_state, function_symbol)

    return function_symbol.symbol_type

def _infer_builtin_print_type(self, node: FunctionCall, scope: SymbolTable) -> LanguageType:
    if len(node.args) != 1:
        raise TypeCheckError(
            self._error_at(node, f"Built-in function 'print' expects 1 argument, got {len(node.args)}")
        )
    arg_type = self._infer_expr_type(node.args[0], scope)
    if arg_type == LanguageType.VOID:
        raise TypeCheckError(
            self._error_at(
                node.args[0],
                "Argument to 'print' must not be Void; a void-returning function cannot be used as an expression",
            )
        )
    return LanguageType.VOID

def _check_function_body(self, function_state: _FunctionState, function_symbol: FunctionSymbol) -> None:
    function_context = _FunctionContext(name=function_state.node.name)
    function_scope = self.global_scope.child_scope()
    for parameter_name, parameter_type in function_symbol.parameters:
        function_scope.define_variable(parameter_name, parameter_type, initialized=True)

    self._active_calls.append(function_state.node.name)
    try:
        self._check_block(function_state.node.body, function_scope, function_context)
    finally:
        self._active_calls.pop()

    function_symbol.symbol_type = function_context.return_type or LanguageType.VOID
    function_state.body_checked = True

```

```

    if (
        function_symbol.symbol_type != LanguageType.VOID
        and not self._definitely_returns(function_state.node.body)
    ):
        raise TypeCheckError(
            self._error_at(
                function_state.node,
                f"Function '{function_state.node.name}' does not return a value on all execution paths",
            )
        )

def _check_uncalled_functions(self) -> None:
    """After all call-site checks, type-check any function whose body was never visited."""
    for name, state in self._functions.items():
        if not state.body_checked:
            try:
                function_symbol = self.global_scope.lookup(name)
            except SymbolTableError:
                continue
            if not isinstance(function_symbol, FunctionSymbol):
                continue
            inferred_params = self._infer_param_types_from_body(state.node)
            function_symbol.parameters = inferred_params
            self._check_function_body(state, function_symbol)

@staticmethod
def _definitely_returns(block: Block) -> bool:
    """Return True if every execution path through block ends in a return statement."""
    for statement in block.statements:
        if isinstance(statement, Return):
            return True
        if isinstance(statement, If) and statement.else_block is not None:
            if (
                TypeChecker._definitely_returns(statement.then_block)
                and TypeChecker._definitely_returns(statement.else_block)
            ):
                return True
    return False

def _infer_param_types_from_body(self, node: FunctionDef) -> list[tuple[str, LanguageType]]:
    """Infer parameter types by scanning the body for expression contexts.

    Any parameter used as an operand of a float operator (+, -, *, /, .) is
    inferred as Float; a parameter used with an integer operator (+, -, *, /,
    ==, !=) is inferred as Integer. Parameters with no type hint default to
    Integer.
    """
    param_names = set(node.params)
    inferred: dict[str, LanguageType] = {}

    def scan_expr(n: ASTNode) -> None:
        if isinstance(n, BinaryOp):
            if n.op in {"+", "-", "*", "/", "==", "!="}:
                for operand in (n.left, n.right):
                    if isinstance(operand, Identifier) and operand.name in param_names:
                        inferred.setdefault(operand.name, LanguageType.INTEGER)
            elif n.op in {"+.", "-.", "*.", "/."}:
                for operand in (n.left, n.right):
                    if isinstance(operand, Identifier) and operand.name in param_names:
                        inferred.setdefault(operand.name, LanguageType.FLOAT)
            scan_expr(n.left)
            scan_expr(n.right)
        elif isinstance(n, FunctionCall):
            for arg in n.args:
                scan_expr(arg)

    def scan_stmt(n: ASTNode) -> None:
        if isinstance(n, Assign):
            scan_expr(n.value)
        elif isinstance(n, Return):

```

```

        scan_expr(n.expr)
    elif isinstance(n, If):
        scan_expr(n.condition)
        scan_block(n.then_block)
    if n.else_block:
        scan_block(n.else_block)
    elif isinstance(n, While):
        scan_expr(n.condition)
        scan_block(n.body)
    elif isinstance(n, Block):
        scan_block(n)
    elif isinstance(n, FunctionCall):
        for arg in n.args:
            scan_expr(arg)

def scan_block(block: Block) -> None:
    for stmt in block.statements:
        scan_stmt(stmt)

scan_block(node.body)
return [(param, inferred.get(param, LanguageType.INTEGER)) for param in node.params]

def _assign_variable_type(self, scope: SymbolTable, node: Assign, value_type: LanguageType) -> None:
    if value_type == LanguageType.VOID:
        raise TypeCheckError(self._error_at(node, f"Cannot assign the result of a void function to '{node.name}'"))
    try:
        symbol = scope.lookup(node.name)
    except SymbolTableError:
        scope.define_variable(node.name, value_type, initialized=True)
        return

    if not isinstance(symbol, VariableSymbol):
        raise TypeCheckError(self._error_at(node, f"'{node.name}' is not a variable"))
    if symbol.symbol_type != value_type:
        raise TypeCheckError(
            self._error_at(
                node,
                f"Cannot assign {value_type.value} to variable '{node.name}' of type {symbol.symbol_type.value}",
            )
        )
    symbol.initialized = True

def _infer_binary_type(
    self,
    node: BinaryOp,
    left_type: LanguageType,
    right_type: LanguageType,
) -> LanguageType:
    # Integer operators: both operands must be Integer
    if node.op in {"+", "-", "*"}:
        if left_type != LanguageType.INTEGER or right_type != LanguageType.INTEGER:
            raise TypeCheckError(
                self._error_at(node, f"Operator '{node.op}' requires two Integer operands")
            )
        return LanguageType.INTEGER

    # Integer division: both operands must be Integer, result is Integer
    if node.op == "/":
        if left_type != LanguageType.INTEGER or right_type != LanguageType.INTEGER:
            raise TypeCheckError(
                self._error_at(node, f"Operator '{node.op}' requires two Integer operands")
            )
        return LanguageType.INTEGER

    # Float operators: both operands must be Float
    if node.op in {"+.", "-.", "*.", "/."}:
        if left_type != LanguageType.FLOAT or right_type != LanguageType.FLOAT:
            raise TypeCheckError(

```



```

        self._error_at(node, f"Operator '{node.op}' requires two Float operands")
    )
    return LanguageType.FLOAT

if node.op in {"==", "!="}:
    if not self._is_numeric(left_type) or not self._is_numeric(right_type):
        raise TypeCheckError(
            self._error_at(
                node,
                f"Operator '{node.op}' requires two arithmetic operands",
            )
        )
    if left_type != right_type:
        raise TypeCheckError(
            self._error_at(
                node,
                f"Operator '{node.op}' requires operands of the same type, got {left_type.value} and {right_type.value}",
            )
        )
    return LanguageType.BOOLEAN

raise TypeCheckError(self._error_at(node, f"Unsupported operator: {node.op}"))

def _lookup_function_symbol(self, node: FunctionCall) -> FunctionSymbol:
    try:
        symbol = self.global_scope.lookup(node.name)
    except SymbolTableError as error:
        raise TypeCheckError(self._error_at(node, str(error))) from error
    if not isinstance(symbol, FunctionSymbol):
        raise TypeCheckError(self._error_at(node, f"'{node.name}' is not a function"))
    return symbol

@staticmethod
def _is_numeric(language_type: LanguageType) -> bool:
    return language_type in {LanguageType.INTEGER, LanguageType.FLOAT}

@staticmethod
def _literal_type(node: Literal) -> LanguageType:
    if isinstance(node.value, bool):
        return LanguageType.BOOLEAN
    if isinstance(node.value, int):
        return LanguageType.INTEGER
    if isinstance(node.value, float):
        return LanguageType.FLOAT
    if isinstance(node.value, str):
        return LanguageType.STRING
    raise TypeCheckError(f"Unsupported literal value: {node.value!r}")

@staticmethod
def _error_at(node: ASTNode, message: str) -> str:
    return f"[line {node.line}, col {node.column}] TypeError: {message}"

```

Listing 8.5: components/interpreter.py

```

from __future__ import annotations

from dataclasses import dataclass
from typing import Callable

from components.ast_nodes import (
    ASTNode,
    Assign,
    BinaryOp,
    Block,
    FunctionCall,
    FunctionDef,
    Identifier,
    If,

```

```

    Literal,
    Program,
    Return,
    While,
)
from components.symbol_table import LanguageType

class InterpreterError(Exception):
    pass

class _ReturnSignal(Exception):
    def __init__(self, value: object) -> None:
        self.value = value

@dataclass
class _FunctionRuntime:
    node: FunctionDef

class _Environment:
    def __init__(self, parent: _Environment | None = None) -> None:
        self.parent = parent
        self.values: dict[str, object] = {}

    def define(self, name: str, value: object) -> None:
        self.values[name] = value

    def get(self, name: str) -> object:
        if name in self.values:
            return self.values[name]
        if self.parent is not None:
            return self.parent.get(name)
        raise InterpreterError(f"Undefined variable: {name}")

    def assign(self, name: str, value: object) -> None:
        if name in self.values:
            self.values[name] = value
            return
        if self.parent is not None and self.parent.contains(name):
            self.parent.assign(name, value)
            return
        self.values[name] = value

    def contains(self, name: str) -> bool:
        if name in self.values:
            return True
        if self.parent is not None:
            return self.parent.contains(name)
        return False

class Interpreter:
    _BUILTIN_PRINT = "print"

    def __init__(self, output_callback: Callable[[str], None] | None = None) -> None:
        self._functions: dict[str, _FunctionRuntime] = {}
        self._output_callback = output_callback or print

    def execute(self, program: Program) -> None:
        environment = _Environment()
        for statement in program.statements:
            if isinstance(statement, FunctionDef):
                self._functions[statement.name] = _FunctionRuntime(node=statement)

        for statement in program.statements:
            if isinstance(statement, FunctionDef):
                continue

```

```

        self._execute_statement(statement, environment)

def _execute_statement(self, node: ASTNode, environment: _Environment) -> None:
    if isinstance(node, Assign):
        value = self._evaluate(node.value, environment)
        environment.assign(node.name, value)
        return

    if isinstance(node, Return):
        raise _ReturnSignal(self._evaluate(node.expr, environment))

    if isinstance(node, If):
        condition = self._evaluate(node.condition, environment)
        if not isinstance(condition, bool):
            raise InterpreterError(self._error_at(node.condition, "If condition must evaluate to Boolean
            "))
        branch = node.then_block if condition else node.else_block
        if branch is not None:
            self._execute_block(branch, _Environment(parent=environment))
        return

    if isinstance(node, While):
        while True:
            condition = self._evaluate(node.condition, environment)
            if not isinstance(condition, bool):
                raise InterpreterError(self._error_at(node.condition, "While condition must evaluate to
                Boolean"))
            if not condition:
                break
            self._execute_block(node.body, _Environment(parent=environment))
        return

    if isinstance(node, Block):
        self._execute_block(node, _Environment(parent=environment))
        return

    if isinstance(node, FunctionCall):
        self._evaluate(node, environment)
        return

    raise InterpreterError(self._error_at(node, f"Unsupported statement node: {type(node).__name__}"))

def _execute_block(self, block: Block, environment: _Environment) -> None:
    for statement in block.statements:
        self._execute_statement(statement, environment)

def _evaluate(self, node: ASTNode, environment: _Environment) -> object:
    if isinstance(node, Literal):
        return node.value

    if isinstance(node, Identifier):
        return environment.get(node.name)

    if isinstance(node, BinaryOp):
        left_value = self._evaluate(node.left, environment)
        right_value = self._evaluate(node.right, environment)
        return self._evaluate_binary(node, left_value, right_value)

    if isinstance(node, FunctionCall):
        return self._call_function(node, environment)

    raise InterpreterError(self._error_at(node, f"Unsupported expression node: {type(node).__name__}"))

def _call_function(self, node: FunctionCall, environment: _Environment) -> object:
    if node.name == self._BUILTIN_PRINT:
        return self._call_builtin_print(node, environment)

    function_runtime = self._functions.get(node.name)
    if function_runtime is None:
        raise InterpreterError(self._error_at(node, f"Undefined function: {node.name}"))

```

```

    if len(node.args) != len(function_runtime.node.params):
        raise InterpreterError(
            self._error_at(
                node,
                f"Function '{node.name}' expects {len(function_runtime.node.params)} argument(s), got {
                    len(node.args)}",
            )
        )

    call_environment = _Environment(parent=environment)
    argument_values = [self._evaluate(argument, environment) for argument in node.args]
    for parameter_name, argument_value in zip(function_runtime.node.params, argument_values, strict=
        False):
        call_environment.define(parameter_name, argument_value)

    try:
        self._execute_block(function_runtime.node.body, call_environment)
    except _ReturnSignal as signal:
        return signal.value
    return None

def _call_builtin_print(self, node: FunctionCall, environment: _Environment) -> None:
    if len(node.args) != 1:
        raise InterpreterError(
            self._error_at(node, f"Built-in function 'print' expects 1 argument, got {len(node.args)}")
        )
    value = self._evaluate(node.args[0], environment)
    self._output_callback(f"{self._format_value(value)} : {self._runtime_type(value).value}")
    return None

def _evaluate_binary(self, node: BinaryOp, left_value: object, right_value: object) -> object:
    if node.op == "+":
        return left_value + right_value
    if node.op == "-":
        return left_value - right_value
    if node.op == "*":
        return left_value * right_value
    if node.op == "/":
        if int(right_value) == 0:
            raise InterpreterError(self._error_at(node, "Division by zero"))
        return int(left_value) // int(right_value)
    if node.op == "+.":
        return float(left_value) + float(right_value)
    if node.op == "-.":
        return float(left_value) - float(right_value)
    if node.op == ".*":
        return float(left_value) * float(right_value)
    if node.op == "/.":
        if float(right_value) == 0.0:
            raise InterpreterError(self._error_at(node, "Division by zero"))
        return float(left_value) / float(right_value)
    if node.op == "==":
        return left_value == right_value
    if node.op == "!=":
        return left_value != right_value
    raise InterpreterError(self._error_at(node, f"Unsupported operator: {node.op}"))

@staticmethod
def _format_value(value: object) -> str:
    if isinstance(value, str):
        return f'"{value}"'
    if isinstance(value, bool):
        return "true" if value else "false"
    return str(value)

@staticmethod
def _runtime_type(value: object) -> LanguageType:
    if isinstance(value, bool):
        return LanguageType.BOOLEAN

```

```

    if isinstance(value, int):
        return LanguageType.INTEGER
    if isinstance(value, float):
        return LanguageType.FLOAT
    if isinstance(value, str):
        return LanguageType.STRING
    return LanguageType.VOID

    @staticmethod
    def _error_at(node: ASTNode, message: str) -> str:
        return f"[line {node.line}, col {node.column}] RuntimeError: {message}"

```

Listing 8.6: components/pipeline.py

```

from __future__ import annotations

from dataclasses import dataclass

from components.ast_printer import ASTPrinter
from components.interpreter import Interpreter
from components.lexica import Lexer
from components.parse_tree_printer import ParseTreePrinter
from components.parser import Parser
from components.symbol_table import SymbolTable
from components.tokens import Token
from components.type_checker import TypeChecker

@dataclass
class PipelineResult:
    tokens: list[Token]
    parse_tree_text: str
    ast_text: str
    checked_scope: SymbolTable
    outputs: list[str]

def run_pipeline(source_code: str) -> PipelineResult:
    tokens = Lexer(source_code).tokenize()
    tree = Parser(tokens).parse()
    parse_tree_text = ParseTreePrinter().print(tree)
    ast_text = ASTPrinter().print(tree)
    checked_scope = TypeChecker().check(tree)

    outputs: list[str] = []
    Interpreter(output_callback=outputs.append).execute(tree)

    return PipelineResult(
        tokens=tokens,
        parse_tree_text=parse_tree_text,
        ast_text=ast_text,
        checked_scope=checked_scope,
        outputs=outputs,
    )

def format_tokens(tokens: list[Token]) -> str:
    lines = []
    for token in tokens:
        lines.append(
            f" {token.token_type.name:<14} {token.lexeme!r:<12} line={token.line} col={token.column}"
        )
    return "\n".join(lines)

def format_stage_output(result: PipelineResult) -> str:
    sections = [
        "=" * 60,
        " STAGE 1 -- TOKENS",
    ]

```

```

    "=" * 60,
    format_tokens(result.tokens),
    "",
    "=" * 60,
    " STAGE 2 -- PARSE TREE",
    "=" * 60,
    result.parse_tree_text,
    "",
    "=" * 60,
    " STAGE 3 -- AST",
    "=" * 60,
    result.ast_text,
    "",
    "=" * 60,
    " STAGE 4 -- TYPE CHECK",
    "=" * 60,
    result.checked_scope.format_table(),
    "",
    "=" * 60,
    " STAGE 5 -- EXECUTION",
    "=" * 60,
    "\n".join(result.outputs) if result.outputs else "(no output)",
]
return "\n".join(sections)

```

A.2 Representative Test File

Listing 8.7: tests/test_pipeline.py

```

from __future__ import annotations

import unittest

from components.interpreter import InterpreterError
from components.lexica import Lexer, LexerError
from components.parser import Parser, ParseError
from components.pipeline import run_pipeline
from components.symbol_table import LanguageType, SymbolTable, SymbolTableError
from components.tokens import TokenType
from components.type_checker import TypeCheckError

# -----
# Lexer
# -----

class LexerTests(unittest.TestCase):
    """Tests for components/lexica.py -- lexical analysis and tokenisation."""

    def test_integer_literal_token(self) -> None:
        tokens = Lexer("42").tokenize()
        self.assertEqual(tokens[0].token_type, TokenType.INT_LITERAL)
        self.assertEqual(tokens[0].lexeme, "42")

    def test_float_literal_token(self) -> None:
        tokens = Lexer("3.14").tokenize()
        self.assertEqual(tokens[0].token_type, TokenType.FLOAT_LITERAL)
        self.assertEqual(tokens[0].lexeme, "3.14")

    def test_bool_literals_produce_bool_literal_tokens(self) -> None:
        tokens = Lexer("true false").tokenize()
        self.assertEqual(tokens[0].token_type, TokenType.BOOL_LITERAL)
        self.assertEqual(tokens[0].lexeme, "true")
        self.assertEqual(tokens[1].token_type, TokenType.BOOL_LITERAL)
        self.assertEqual(tokens[1].lexeme, "false")

    def test_keywords_produce_correct_token_types(self) -> None:

```

```

tokens = Lexer("if else while def return").tokenize()
expected = [
    TokenType.IF, TokenType.ELSE, TokenType.WHILE, TokenType.DEF, TokenType.RETURN,
]
actual = [t.token_type for t in tokens if t.token_type != TokenType.EOF]
self.assertEqual(actual, expected)

def test_identifier_not_confused_with_keyword(self) -> None:
    tokens = Lexer("iffy whileloop print").tokenize()
    self.assertEqual(tokens[0].token_type, TokenType.IDENTIFIER)
    self.assertEqual(tokens[1].token_type, TokenType.IDENTIFIER)
    self.assertEqual(tokens[2].token_type, TokenType.IDENTIFIER)

def test_two_character_operators(self) -> None:
    tokens = Lexer("== !=").tokenize()
    self.assertEqual(tokens[0].token_type, TokenType.EQUAL_EQUAL)
    self.assertEqual(tokens[1].token_type, TokenType.BANG_EQUAL)

def test_float_dot_operators_tokenised(self) -> None:
    tokens = Lexer("+. -. *. /.").tokenize()
    expected = [TokenType.PLUS_DOT, TokenType.MINUS_DOT, TokenType.STAR_DOT, TokenType.SLASH_DOT]
    actual = [t.token_type for t in tokens if t.token_type != TokenType.EOF]
    self.assertEqual(actual, expected)

def test_float_literal_not_confused_with_dot_operator(self) -> None:
    # "3.14" must produce FLOAT_LITERAL, not INT_LITERAL followed by something
    tokens = Lexer("3.14").tokenize()
    self.assertEqual(tokens[0].token_type, TokenType.FLOAT_LITERAL)
    self.assertEqual(tokens[0].lexeme, "3.14")

def test_source_position_tracked_across_lines(self) -> None:
    tokens = Lexer("x\ny").tokenize()
    self.assertEqual(tokens[0].line, 1)
    self.assertEqual(tokens[1].line, 2)

def test_unexpected_character_raises_lexer_error(self) -> None:
    with self.assertRaises(LexerError):
        Lexer("@").tokenize()

def test_lone_exclamation_raises_lexer_error(self) -> None:
    with self.assertRaises(LexerError):
        Lexer("!5").tokenize()

def test_unterminated_string_raises_lexer_error(self) -> None:
    with self.assertRaises(LexerError):
        Lexer('"hello').tokenize()

# -----
# Symbol table
# -----

class SymbolTableTests(unittest.TestCase):
    """Tests for components/symbol_table.py -- scoped symbol table and semantic types."""

    def test_define_variable_and_lookup(self) -> None:
        table = SymbolTable()
        table.define_variable("x", LanguageType.INTEGER, initialized=True)
        symbol = table.lookup("x")
        self.assertEqual(symbol.symbol_type, LanguageType.INTEGER)

    def test_duplicate_variable_definition_raises_error(self) -> None:
        table = SymbolTable()
        table.define_variable("x", LanguageType.INTEGER)
        with self.assertRaises(SymbolTableError):
            table.define_variable("x", LanguageType.FLOAT)

    def test_duplicate_function_definition_raises_error(self) -> None:
        table = SymbolTable()
        table.define_function("f", LanguageType.INTEGER, [])

```

```

        with self.assertRaises(SymbolTableError):
            table.define_function("f", LanguageType.FLOAT, [])

def test_lookup_in_parent_scope(self) -> None:
    parent = SymbolTable()
    parent.define_variable("x", LanguageType.INTEGER, initialized=True)
    child = parent.child_scope()
    symbol = child.lookup("x")
    self.assertEqual(symbol.symbol_type, LanguageType.INTEGER)

def test_undefined_symbol_raises_error(self) -> None:
    table = SymbolTable()
    with self.assertRaises(SymbolTableError):
        table.lookup("z")

def test_format_table_contains_variable_row(self) -> None:
    table = SymbolTable()
    table.define_variable("counter", LanguageType.INTEGER, initialized=True)
    output = table.format_table()
    self.assertIn("counter", output)
    self.assertIn("variable", output)
    self.assertIn("Integer", output)

def test_format_table_contains_function_row(self) -> None:
    table = SymbolTable()
    table.define_function("add", LanguageType.INTEGER, [("a", LanguageType.INTEGER)])
    output = table.format_table()
    self.assertIn("add", output)
    self.assertIn("function", output)

# -----
# Parser
# -----

class ParserTests(unittest.TestCase):
    """Tests for components/parser.py -- recursive-descent parsing."""

    def _parse(self, source: str):
        return Parser(Lexer(source).tokenize()).parse()

    def test_missing_semicolon_raises_parse_error(self) -> None:
        with self.assertRaises(ParseError):
            self._parse("x = 5")

    def test_unclosed_brace_raises_parse_error(self) -> None:
        with self.assertRaises(ParseError):
            self._parse("if (true) { x = 1;")

    def test_operator_precedence_mul_before_add(self) -> None:
        # 2 + 3 * 4 must be 14, not 20
        result = run_pipeline("x = 2 + 3 * 4; print(x);")
        self.assertEqual(result.outputs, ["14 : Integer"])

    def test_operator_left_associativity(self) -> None:
        # 10 - 3 - 2 must be 5 (left-assoc), not 9
        result = run_pipeline("x = 10 - 3 - 2; print(x);")
        self.assertEqual(result.outputs, ["5 : Integer"])

    def test_parentheses_override_precedence(self) -> None:
        # (2 + 3) * 4 must be 20
        result = run_pipeline("x = (2 + 3) * 4; print(x);")
        self.assertEqual(result.outputs, ["20 : Integer"])

    def test_float_operator_precedence_mul_before_add(self) -> None:
        # 1.0 +. 2.0 *. 3.0 must be 7.0 (*. binds tighter than +.)
        result = run_pipeline("x = 1.0 +. 2.0 *. 3.0; print(x);")
        self.assertEqual(result.outputs, ["7.0 : Float"])

    def test_pipeline_exposes_parse_tree_and_ast(self) -> None:

```



```

    result = run_pipeline("x = 5; print(x);")
    self.assertIn("program", result.parse_tree_text)
    self.assertIn("assignment", result.parse_tree_text)
    self.assertIn("additive", result.parse_tree_text)
    self.assertIn("term", result.parse_tree_text)
    self.assertIn("factor", result.parse_tree_text)
    self.assertIn("function_call", result.parse_tree_text)
    self.assertIn("Program", result.ast_text)
    self.assertIn("Assign", result.ast_text)
    self.assertIn("FunctionCall 'print'", result.ast_text)

# -----
# Type checker
# -----

class TypeCheckerTests(unittest.TestCase):
    """Tests for components/type_checker.py -- static type inference and checking."""

    def test_integer_arithmetic_stays_integer(self) -> None:
        result = run_pipeline("x = 3 + 4; print(x);")
        self.assertEqual(result.outputs, ["7 : Integer"])

    def test_division_always_produces_integer(self) -> None:
        # integer / integer -> Integer (no implicit float conversion)
        result = run_pipeline("x = 10 / 2; print(x);")
        self.assertEqual(result.outputs, ["5 : Integer"])

    def test_float_dot_operators_produce_float(self) -> None:
        result = run_pipeline("x = 1.0 +. 2.5; print(x);")
        self.assertEqual(result.outputs, ["3.5 : Float"])

    def test_float_subtraction_produces_float(self) -> None:
        result = run_pipeline("x = 5.0 -. 1.5; print(x);")
        self.assertEqual(result.outputs, ["3.5 : Float"])

    def test_float_multiplication_produces_float(self) -> None:
        result = run_pipeline("x = 2.0 *. 3.0; print(x);")
        self.assertEqual(result.outputs, ["6.0 : Float"])

    def test_float_division_produces_float(self) -> None:
        result = run_pipeline("x = 9.0 /. 4.0; print(x);")
        self.assertEqual(result.outputs, ["2.25 : Float"])

    def test_integer_with_float_op_raises_type_error(self) -> None:
        # +. requires Float operands; integers must not be accepted
        with self.assertRaises(TypeError):
            run_pipeline("x = 2 +. 3; print(x);")

    def test_mixed_int_float_addition_raises_type_error(self) -> None:
        # Integer + Float is no longer valid; must use +.
        with self.assertRaises(TypeError):
            run_pipeline("x = 1 + 2.5; print(x);")

    def test_string_variable_type_inferred(self) -> None:
        result = run_pipeline('x = "hello"; print(x);')
        self.assertEqual(result.outputs, ["'hello" : String'])

    def test_boolean_literal_type_inferred(self) -> None:
        result = run_pipeline("x = true; print(x);")
        self.assertEqual(result.outputs, ["true : Boolean"])

    def test_boolean_equality_raises_type_error(self) -> None:
        # Boolean == Boolean is not allowed; == only accepts arithmetic operands
        with self.assertRaises(TypeError):
            run_pipeline('x = true; if (x == false) { print("then"); } else { print("else"); }')

    def test_mixed_int_float_equality_raises_type_error(self) -> None:
        # == and != must not accept mixed Integer/Float (no overloading)
        with self.assertRaises(TypeError):

```

```

        run_pipeline("x = 5; y = 3.14; if (x == y) { print(x); }")

def test_mixed_int_float_inequality_raises_type_error(self) -> None:
    with self.assertRaises(TypeError):
        run_pipeline("x = 5; y = 3.14; if (x != y) { print(x); }")

def test_arithmetic_on_string_raises_type_error(self) -> None:
    with self.assertRaises(TypeError):
        run_pipeline('x = "a" + 1;')

def test_if_non_boolean_condition_raises_type_error(self) -> None:
    with self.assertRaises(TypeError):
        run_pipeline("if (1) { x = 2; }")

def test_while_non_boolean_condition_raises_type_error(self) -> None:
    with self.assertRaises(TypeError):
        run_pipeline("x = 0; while (x) { x = x + 1; }")

def test_assignment_type_mismatch_raises_type_error(self) -> None:
    with self.assertRaises(TypeError):
        run_pipeline('x = 5; x = "hello";')

def test_undefined_variable_raises_type_error(self) -> None:
    with self.assertRaises(TypeError):
        run_pipeline("print(z);")

def test_function_wrong_arg_count_raises_type_error(self) -> None:
    with self.assertRaises(TypeError):
        run_pipeline("def f(a) { return a; } f(1, 2);")

def test_function_arg_type_mismatch_raises_type_error(self) -> None:
    # first call infers a: Integer; second call with String must fail
    with self.assertRaises(TypeError):
        run_pipeline('def f(a) { return a; } f(1); f("hello");')

def test_uncalled_function_body_type_error_is_caught(self) -> None:
    # type error inside body must be detected even if function is never called
    with self.assertRaises(TypeError):
        run_pipeline('def bad() { y = "hello" + 1; }')

def test_valid_uncalled_function_does_not_raise(self) -> None:
    # a well-typed but uncalled function should not raise
    result = run_pipeline("def add(a, b) { return a + b; }")
    self.assertEqual(result.outputs, [])

# -----
# Integration (full pipeline)
# -----

class IntegrationTests(unittest.TestCase):
    """End-to-end tests through the full pipeline."""

    # --- Unary minus ---

    def test_unary_minus_integer(self) -> None:
        result = run_pipeline("x = -5; print(x);")
        self.assertEqual(result.outputs, ["-5 : Integer"])

    def test_unary_minus_float(self) -> None:
        result = run_pipeline("y = -. 3.14; print(y);")
        self.assertEqual(result.outputs, ["-3.14 : Float"])

    def test_unary_minus_dot_variable(self) -> None:
        result = run_pipeline("x = 1.5; y = -. x; print(y);")
        self.assertEqual(result.outputs, ["-1.5 : Float"])

    def test_unary_minus_variable(self) -> None:
        result = run_pipeline("x = 10; y = -x; print(y);")
        self.assertEqual(result.outputs, ["-10 : Integer"])

```

```

def test_unary_minus_inside_expression(self) -> None:
    result = run_pipeline("x = 3 + -2; print(x);")
    self.assertEqual(result.outputs, ["1 : Integer"])

# --- Control flow ---

def test_if_then_branch_executes(self) -> None:
    result = run_pipeline(
        'x = 5; if (x == 5) { print("yes"); } else { print("no"); }'
    )
    self.assertEqual(result.outputs, ['"yes" : String'])

def test_if_else_branch_executes_when_condition_false(self) -> None:
    result = run_pipeline(
        'x = 0; if (x == 5) { print("yes"); } else { print("no"); }'
    )
    self.assertEqual(result.outputs, ['"no" : String'])

def test_while_loop_executes_repeatedly(self) -> None:
    result = run_pipeline("x = 3; while (x != 0) { print(x); x = x - 1; }")
    self.assertEqual(result.outputs, ["3 : Integer", "2 : Integer", "1 : Integer"])

# --- Division by zero ---

def test_integer_division_by_zero_raises_error(self) -> None:
    with self.assertRaises(InterpreterError):
        run_pipeline("x = 10 / 0; print(x);")

def test_float_division_by_zero_raises_error(self) -> None:
    with self.assertRaises(InterpreterError):
        run_pipeline("x = 10.0 /. 0.0; print(x);")

# --- Functions ---

def test_function_integer_return(self) -> None:
    result = run_pipeline("def square(n) { return n * n; } print(square(7));")
    self.assertEqual(result.outputs, ["49 : Integer"])

def test_function_type_inference(self) -> None:
    result = run_pipeline(
        "def add(a, b) { return a +. b; } result = add(2.0, 3.5); print(result);"
    )
    self.assertEqual(result.outputs, ["5.5 : Float"])
    self.assertIn("result variable Float", result.checked_scope.format_table())

def test_function_value_parameter_passing(self) -> None:
    # Modifying a parameter inside a function must not affect the caller's variable
    result = run_pipeline(
        "def inc(a) { a = a + 1; return a; } x = 10; y = inc(x); print(x); print(y);"
    )
    self.assertEqual(result.outputs, ["10 : Integer", "11 : Integer"])

def test_nested_function_calls(self) -> None:
    result = run_pipeline(
        "def add(a, b) { return a + b; } print(add(add(1, 2), 3));"
    )
    self.assertEqual(result.outputs, ["6 : Integer"])

# --- print() with all four types ---

def test_print_integer(self) -> None:
    result = run_pipeline("print(42);")
    self.assertEqual(result.outputs, ["42 : Integer"])

def test_print_float(self) -> None:
    result = run_pipeline("print(3.14);")
    self.assertEqual(result.outputs, ["3.14 : Float"])

def test_print_boolean(self) -> None:

```

```

    result = run_pipeline("print(true);")
    self.assertEqual(result.outputs, ["true : Boolean"])

def test_print_string(self) -> None:
    result = run_pipeline('print("plc");')
    self.assertEqual(result.outputs, ["plc" : String'])

# --- Boolean expressions as values ---

def test_comparison_result_assigned_to_variable(self) -> None:
    result = run_pipeline("x = 5; y = 5; flag = (x == y); print(flag);")
    self.assertEqual(result.outputs, ["true : Boolean"])

def test_comparison_result_printed_directly(self) -> None:
    result = run_pipeline("x = 10; print(x != 0);")
    self.assertEqual(result.outputs, ["true : Boolean"])

def test_comparison_result_false(self) -> None:
    result = run_pipeline("a = 1; b = 2; eq = (a == b); print(eq);")
    self.assertEqual(result.outputs, ["false : Boolean"])

# --- Reassignment ---

def test_variable_can_be_reassigned_same_type(self) -> None:
    result = run_pipeline("x = 1; x = 2; x = 3; print(x);")
    self.assertEqual(result.outputs, ["3 : Integer"])

def test_scope_isolation_if_block(self) -> None:
    # Variable defined inside an if-block must not be visible outside it
    with self.assertRaises(TypeError):
        run_pipeline(
            "x = 1; if (x == 1) { inner = 99; } print(inner);"
        )

def test_string_equality_raises_type_error(self) -> None:
    # == is only valid between arithmetic (Integer/Float) operands
    with self.assertRaises(TypeError):
        run_pipeline('a = "hello"; b = "world"; if (a == b) { print(a); }')

def test_integer_floor_division(self) -> None:
    # 7 / 2 must be 3 (floor), not 3.5
    result = run_pipeline("x = 7 / 2; print(x);")
    self.assertEqual(result.outputs, ["3 : Integer"])

def test_void_function_assignment_raises_type_error(self) -> None:
    # Assigning the result of a void function (no return) must be a type error
    with self.assertRaises(TypeError):
        run_pipeline("def greet() { print(42); } x = greet();")

def test_builtin_print_assignment_raises_type_error(self) -> None:
    with self.assertRaises(TypeError):
        run_pipeline("x = print(42);")

def test_print_void_function_raises_type_error(self) -> None:
    # print(greet()) where greet() is void must be rejected -- Void is not a printable value
    with self.assertRaises(TypeError):
        run_pipeline("def greet() { print(42); } print(greet());")

def test_non_total_function_raises_type_error(self) -> None:
    # A function with a return only in some branches must be rejected
    with self.assertRaises(TypeError):
        run_pipeline("def f(a) { if (a == 1) { return 5; } } x = f(1); print(x);")

if __name__ == "__main__":
    unittest.main()

```